CIS*3110 Operating Systems

Week 13

Lecture #21 FSCK, Journaling, LFS

Prof. Yulia Kotseruba

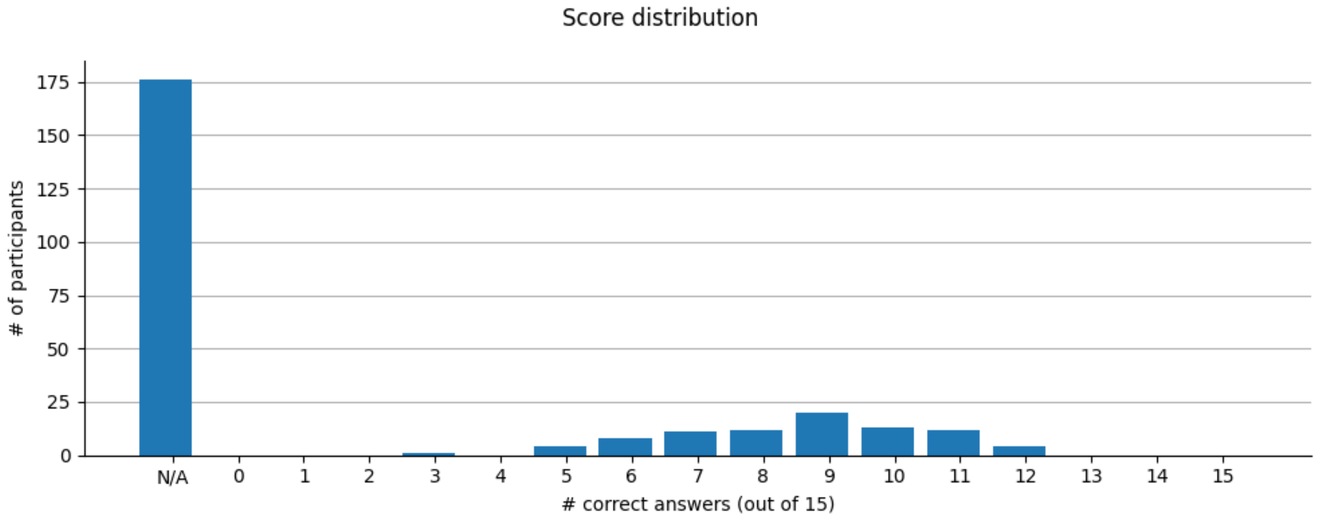
March 31, 2026

Quiz 19

```
int main(){
    int f1 = open("file.txt", O_RDWR|O_CREAT|O_TRUNC, 0444);
    int f2 = dup(f1);
    int f3 = dup(f2);
    write(f1, "A", 1);
    write(f2, "B", 1);
    write(f3, "C", 1);
    close(f1); close(f2); close(f3);
}
```

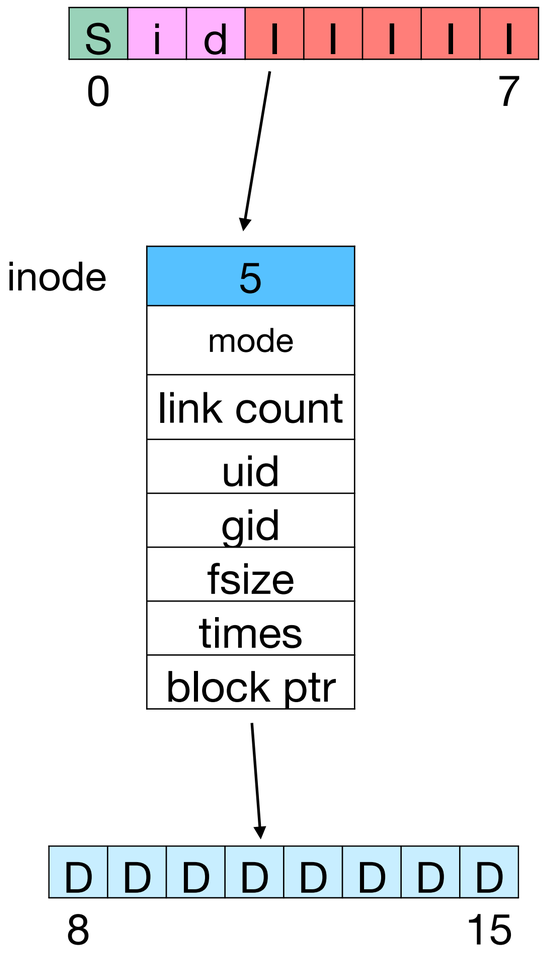
- **56%** average (down from 58% in quiz 18)
- 3 hardest questions:
  - (22%) What will be the contents of the file after this code executes? a) ABC b) C d) BC d) none of the above
  - (33%) In the given memory system, what is the number of frames and pages? a) frames > pages b) frames < pages c) frames = pages d) not enough data
  - (52%) Unlike hard links, symbolic links ____ a) can be cyclical b) can be made to files c) have an inode d) have permissions

```
16-bit virtual address
16KB (2^14) RAM
1KB (2^10) pages
```



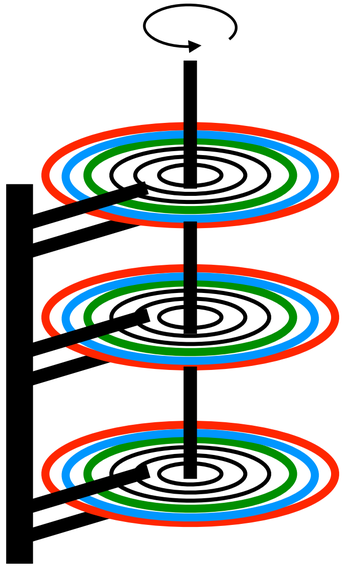
Recap

- very simple file system (vsfs)
- **blocks** - fixed sized units of memory on hard drive
  - hold data and metadata
- Metadata structures
  - **superblock** - metadata about file system
  - **bitmaps** - info about which data blocks and inodes are free
  - **inodes** - contain info for files and directories
    - direct and indirect pointers to data blocks on hard drive



Recap

- vsfs is **very simple** and **very inefficient**
  - basic file operations require **a lot of expensive I/O**
  - Problem: data is spread across the HD
    - on HD writing to random locations is the worst possible scenario
- Solution: Fast File System (FFS)
  - same metadata structures as vsfs
  - **disk-aware allocation policy**
    - ***keep related stuff together and unrelated stuff apart***
    - organize files and directories into **cylinder groups**
      - minimize positioning latency
      - maximize contiguous reads from the same track



Crash consistency

- What happens to I/O in progress when OS crashes?
- Problem #1: potential **corruption** of file system
- Problem #2: **data loss**
- Solution #1: file system checker (**fsck**)
- Solution #2: **journaling**

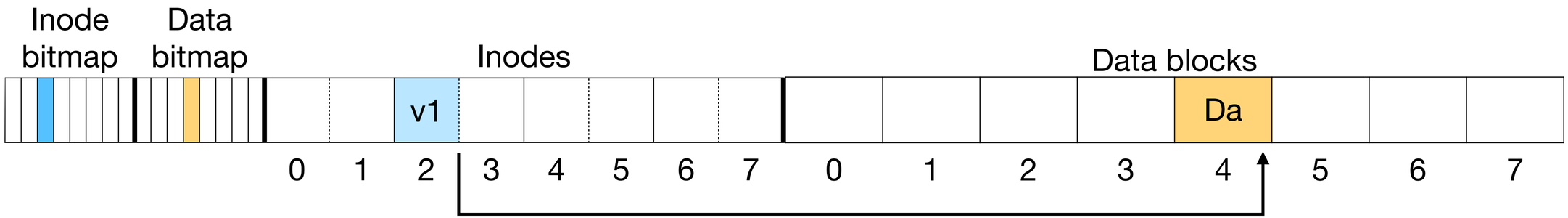
Extending files

- Previous workloads
  - random reads/writes - test average positioning time
  - sequential reads/writes - test throughput
- New test for **crash robustness**
  - appending a data block to one file
    - open a file
    - call lseek() to move the offset to the end of file
    - write 4KB
    - close the file

data block 4KB

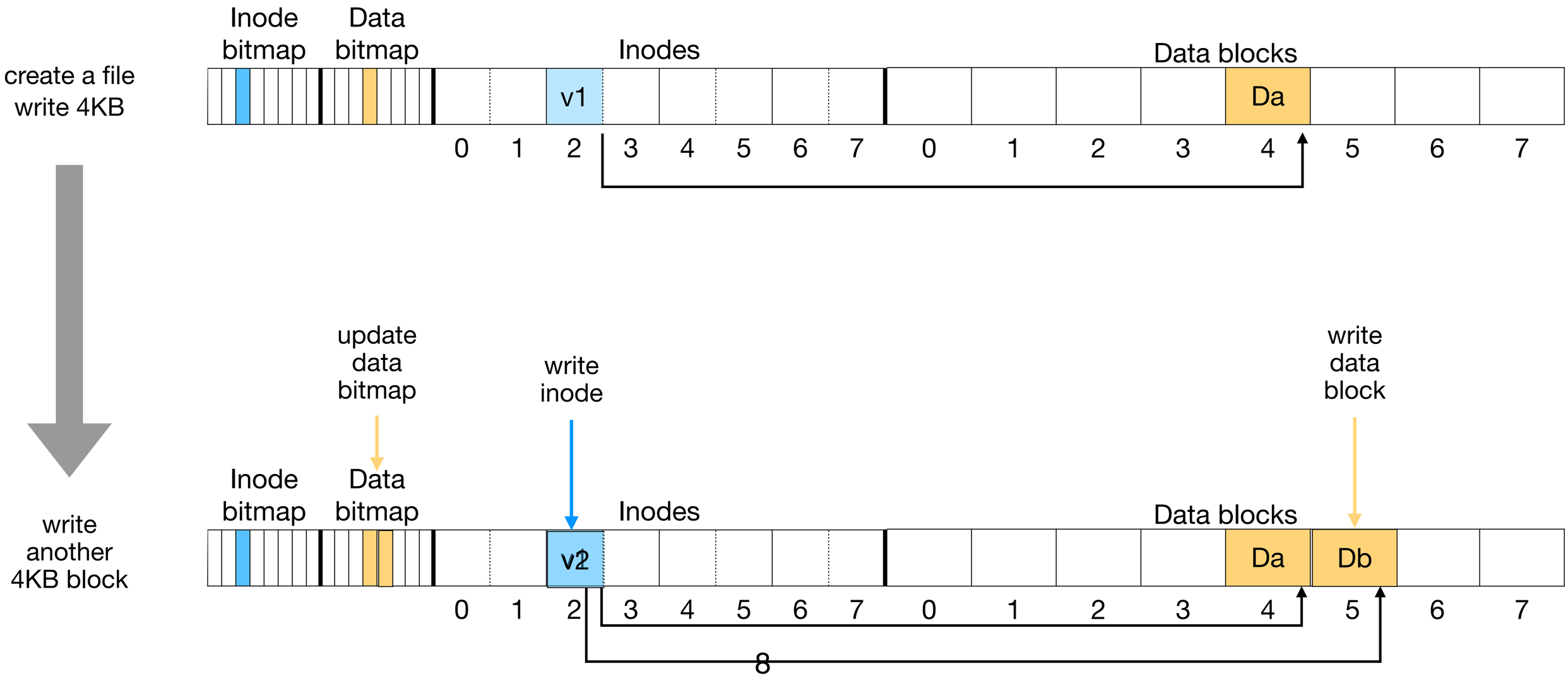
Extending files

create a file
write 4KB



write
another
4KB block

Extending files

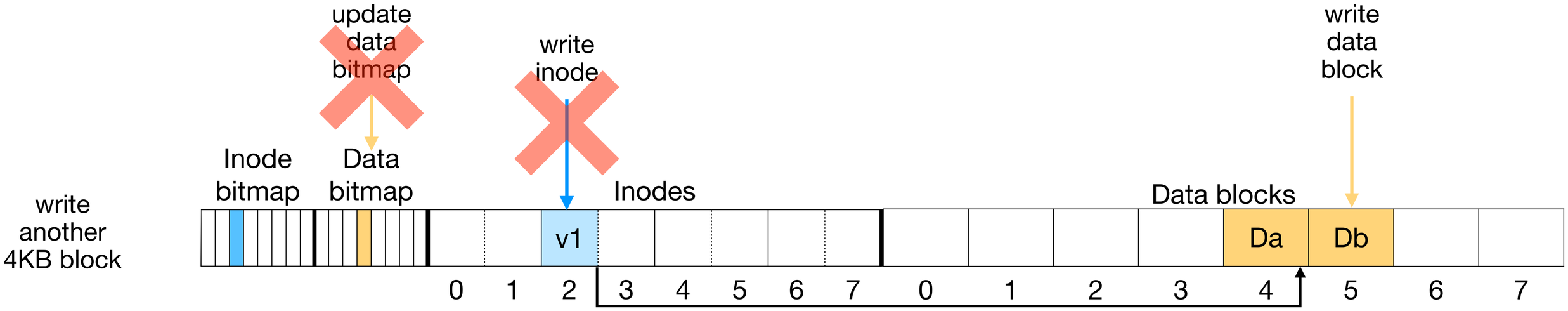


Extending files

What if only the data block is written?

data loss - data is on HD but there is no pointer to it

consistency - OK



bitmap still shows the block as free

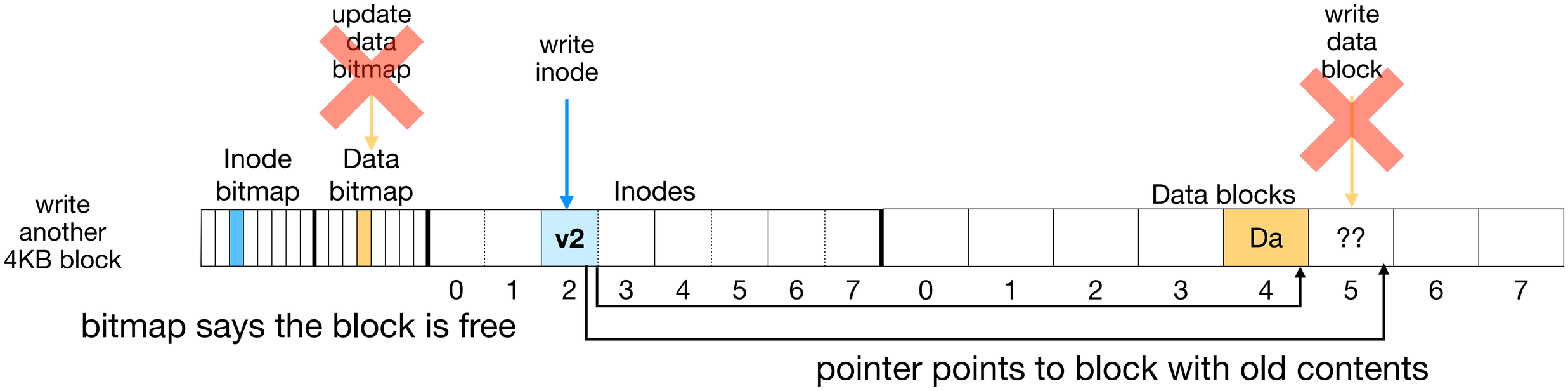
there is no pointer from inode

Extending files

What if only the inode is written?

data loss - pointer from inode is to unwritten block with garbage (whatever was there before)

consistency - inode has a link to data but bitmap says the block is free

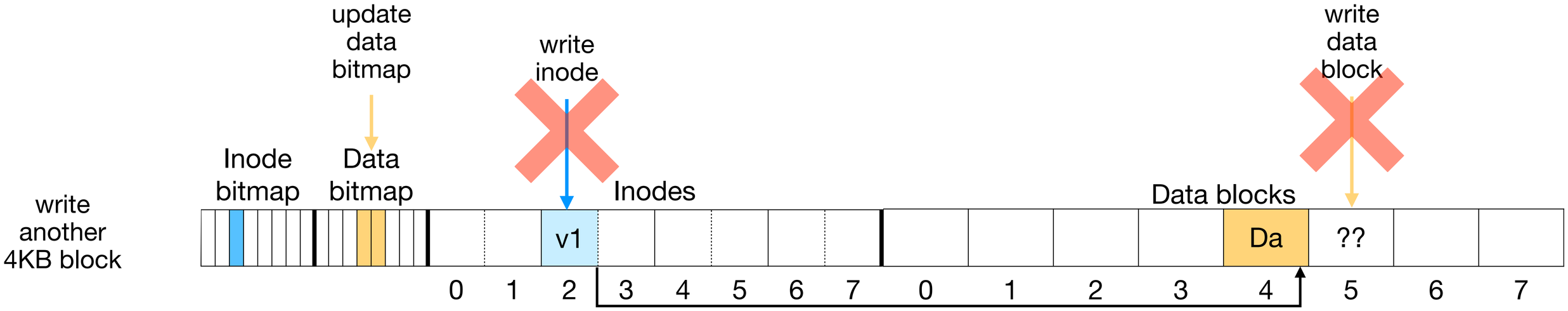


Extending files

What if only bitmap is written?

data loss

consistency



bitmap shows the block as used

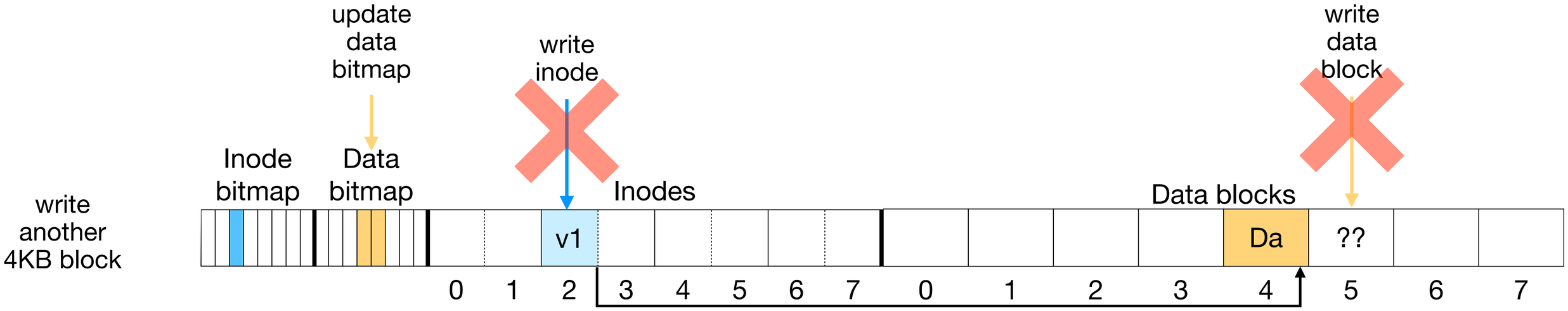
but there is no pointer and no data written

Extending files

What if only bitmap is written?

data loss - data was never written

consistency - block 5 will never be used again



bitmap shows the block as used

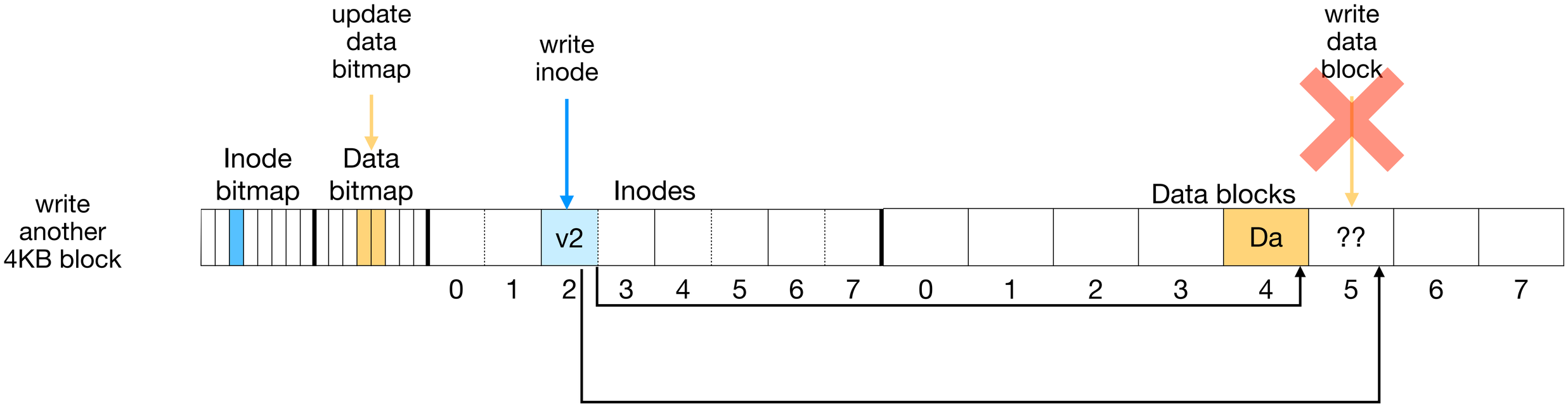
but there is no pointer and no data written

Extending files

What if inode and bitmap are written but data is not?

data loss

consistency

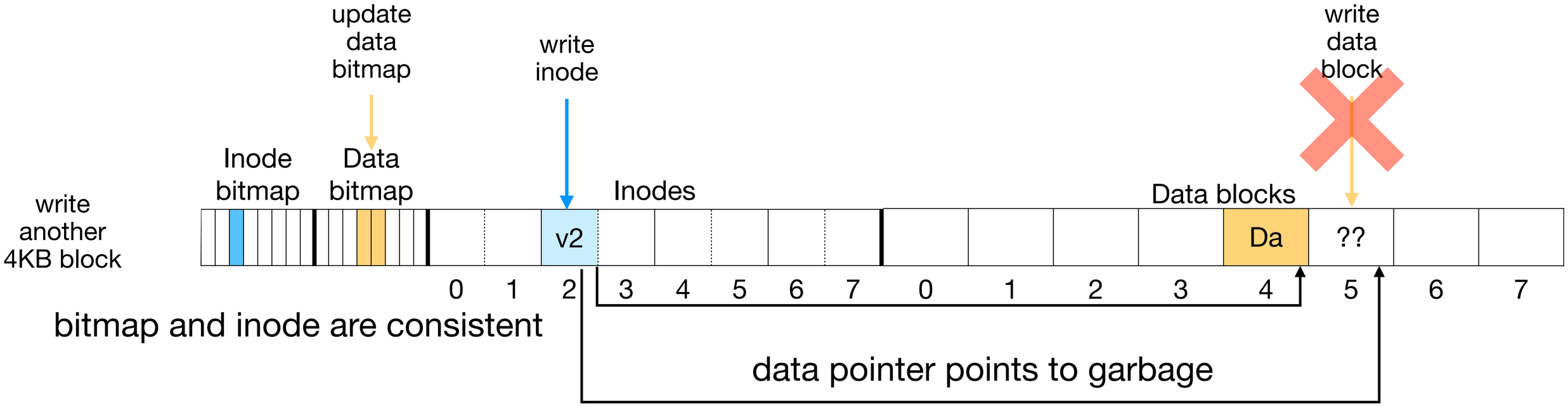


Extending files

What if inode and bitmap are written but data is not?

data loss - pointer from inode is to unwritten block with garbage (whatever was there before)

consistency - OK

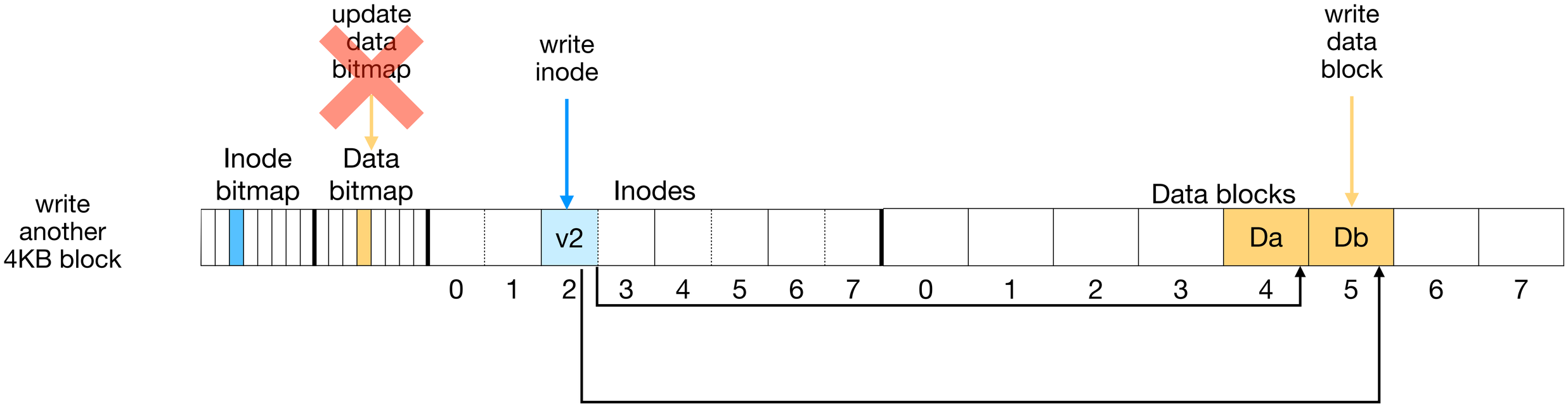


Extending files

What if inode and data are written but bitmap is not?

data loss

consistency

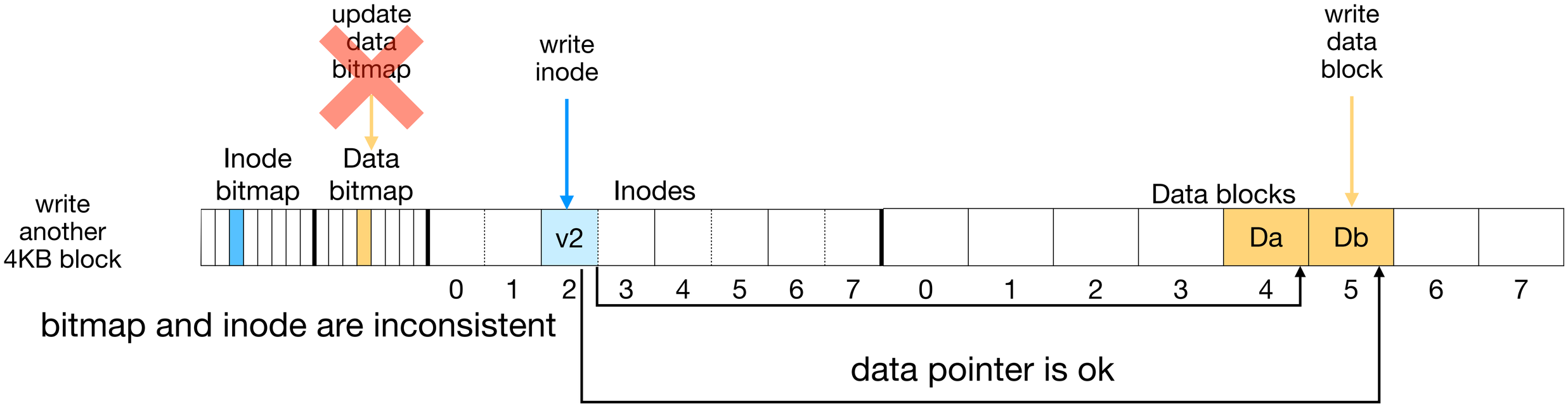


Extending files

What if inode and data are written but bitmap is not?

no data loss

consistency - bitmap shows block as free when there is a pointer to it

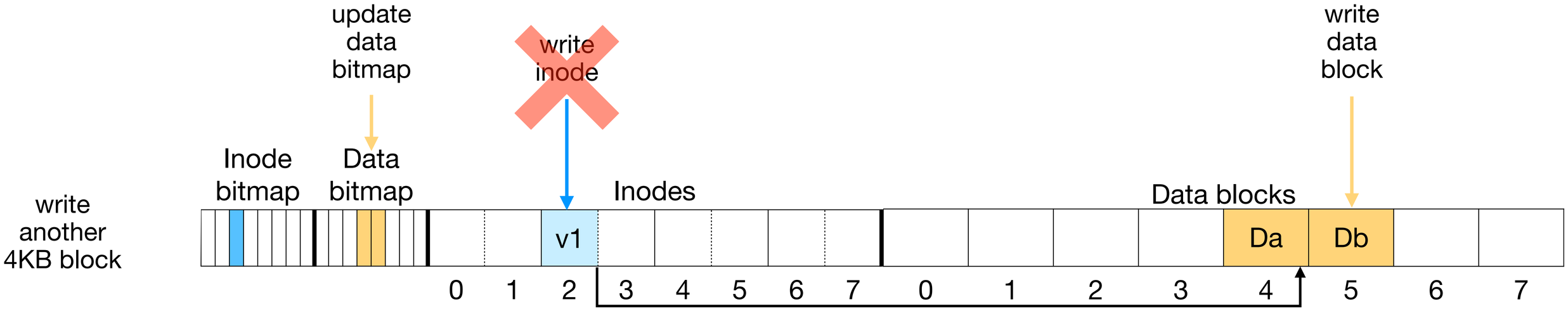


Extending files

What if bitmap and data are written but inode is not?

data loss

consistency

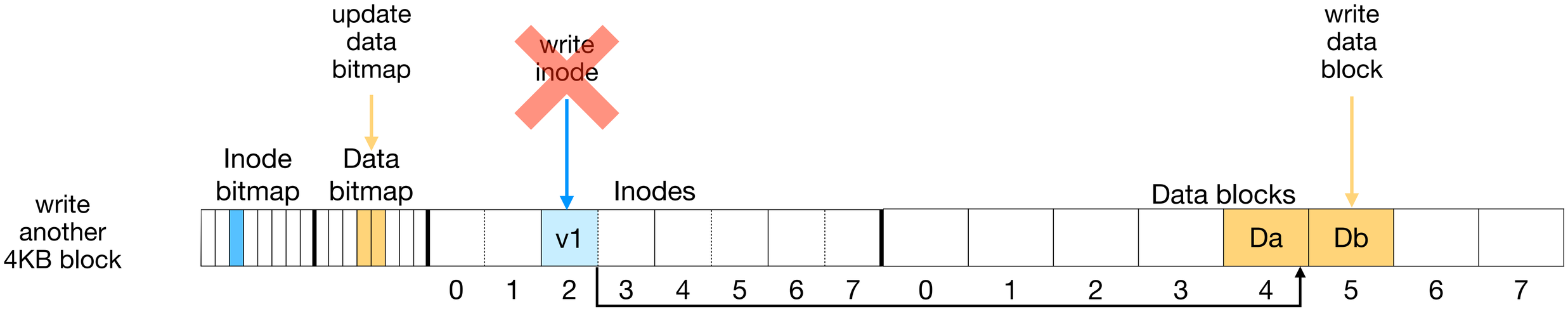


Extending files

What if bitmap and data are written but inode is not?

data loss - there is no pointer from inode to the written data block

consistency - bitmap shows block as used when there is no pointer to it



bitmap and inode are inconsistent

Crash consistency

- aka consistent-update problem
- I/O operations are **not atomic**
  - e.g. writing to a file involves 3 updates
    - inode, bitmap, data block
- Crashes can happen at any time
- Solution #1: **fsck** - let the crash happen and fix later
- Solution #2: **journaling** aka write-ahead logging

fsck

- A UNIX file system checker utility
- A **passive** approach to file system crash consistency
  - Runs after a crash happens when **rebooting**
  - **Finds** and **repairs** file system **inconsistencies**
  - Assumes no other file system activity

fsck phases

- **Superblock** - check if superblock is reasonable
  - FS size > number of allocated blocks
  - finds corrupt superblocks and replaces them with good copies
- **Free blocks** - scan inodes, direct/indirect blocks
  - restore allocation bitmaps from inodes
  - i.e. inconsistencies are resolved in favor of inodes

fsck phases

- **Inode state** - check inode for corruption
  - e.g. fields are valid
  - delete inode if corrupted and update bitmap
- **Inode links** - check link counts of inodes
  - scan through directory tree starting from root
  - rebuild all links and compare to the old counts
  - fix inode link counts
  - allocated nodes without parent directories go to lost+found

fsck phases

- **Duplicate pointers** - several inodes refer to the same block
  - remove corrupted inodes
  - if all inodes are good -> duplicate data and update pointers
- **Bad block pointers** - check the values of data pointer
  - bad pointers have wrong addresses (beyond the partition size)
  - remove bad pointers and update the inode
- **Directory checks** - integrity checks on the directory inodes
  - make sure every directory has . and ..
  - every inode is allocated
  - no cycles in the directory tree

fsck implementation

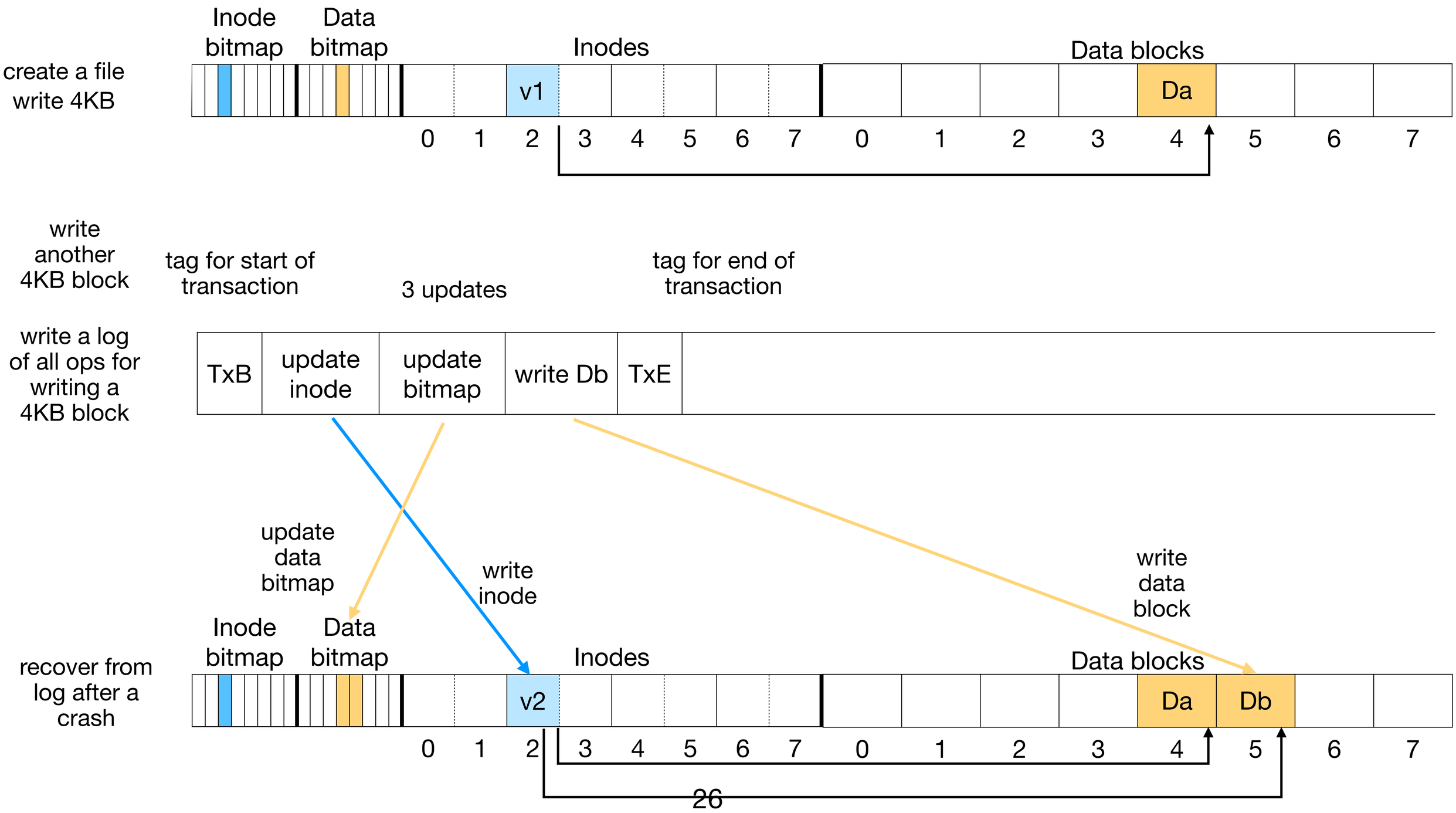
- Has to be designed for each file system
  - Relies on internal data structures
- **Is VERY SLOW**
  - scanning the whole disk takes hours
  - cannot use computer in the meantime

Journaling

- aka write-ahead logging
- Allocate space for a **log** on the hard drive
  - e.g. after the superblock
- When updating the disk
  - first write the list of operations in the log
  - then update the data structures on the disk
- **Use the log to recover** from the crash
  - instead of scanning the whole disk
  - **active** approach to recovery

allocate space for the log in a dedicated location, e.g. after the superblock

Superb lock	Journal	Group 0	Group 1	…	Group N


create a file
write 4KB
Inode bitmap
Data bitmap
Inodes
Data blocks
v1
Da
0 1 2 3 4 5 6 7 0 1 2 3 4 5 6 7
write another 4KB block
tag for start of transaction
3 updates
tag for end of transaction
write a log of all ops for writing a 4KB block
TxB
update inode
update bitmap
write Db
TxE
update data bitmap
write inode
write data block
recover from log after a crash
Inode bitmap
Data bitmap
Inodes
Data blocks
v2
Da
Db
0 1 2 3 4 5 6 7 0 1 2 3 4 5 6 7

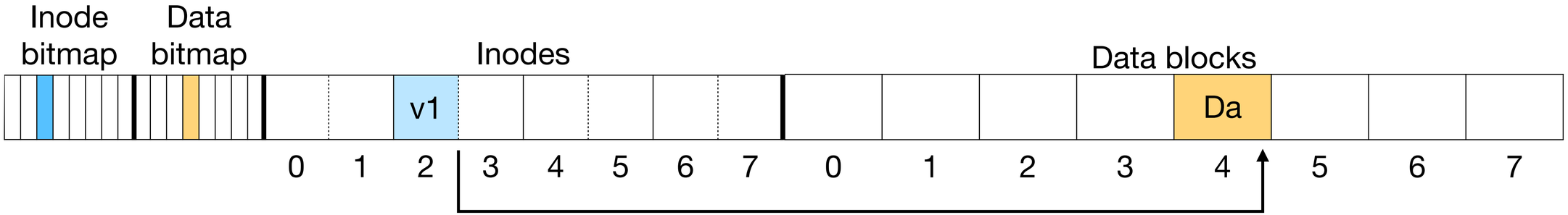
Data Journaling

- **Journal write** - write the transaction to the journal
  - transaction-begin block
  - pending data and metadata updates
  - transaction-end block
  - wait for the writes to complete
- **Checkpoint** - write the pending metadata and data updates to the disk

Data Journaling

- What if crash happens **during journal write?**
  - data and metadata may be large
  - disk has its own scheduling and may write them in any order
  - transaction may become invalid (e.g. corrupted data)

create a file
write 4KB



What if crash happens during the journal write?

The order of writes is not guaranteed to be sequential

e.g. data update is scheduled last and crash happens before it is written

write
another
4KB block

write a log
of all ops for
writing a
4KB block

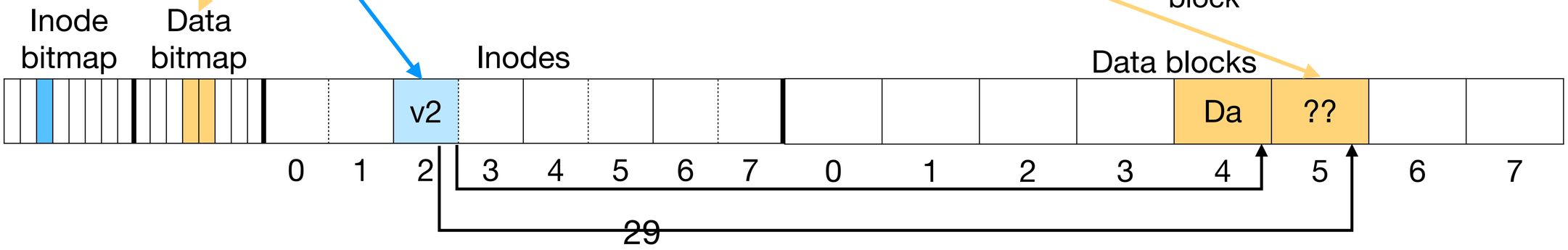
this looks like a valid transaction but it
will write garbage to disk on recovery

update
data
bitmap

write
inode

write
data
block

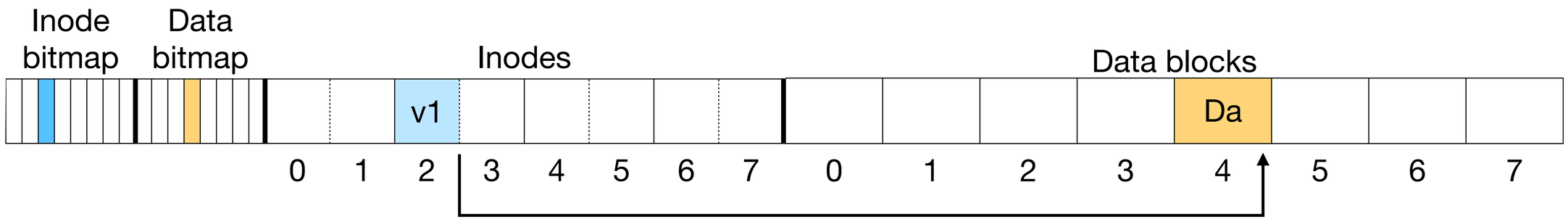
recover from
log after a
crash



Data Journaling

- What if crash happens during journal write?
  - data and metadata may be large
  - disk has its own scheduling and may write them in any order
  - transaction may become invalid (e.g. corrupted data)
- Solution: **two-phase write**
  - write the all blocks except last
  - write the last block
    - writing a block of 512-bytes is **atomic**
      - it will either write or not, will never be half-written

create a file
write 4KB



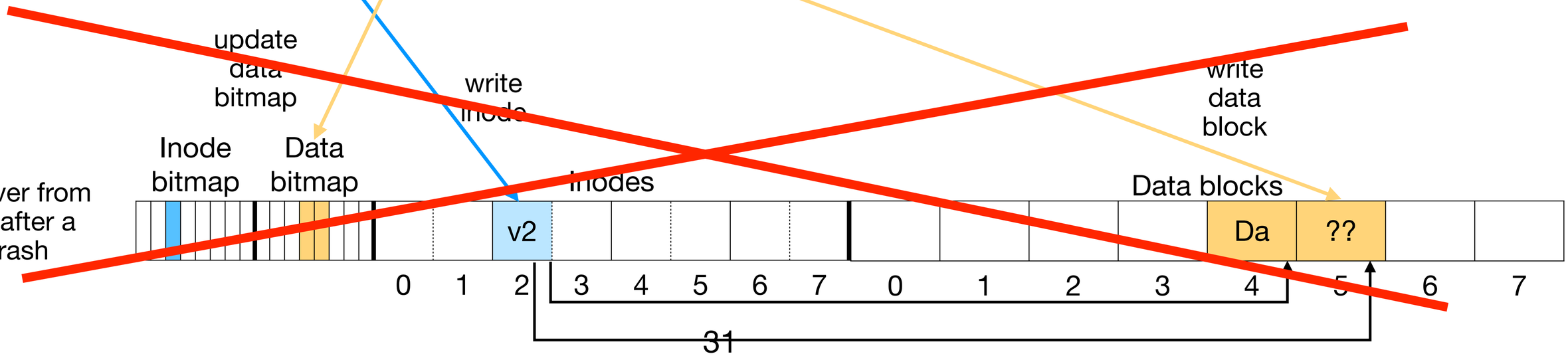
write another 4KB block

Now if a crash happens during the write

the transaction-end block will not be written and recovery will not be done

write a log of all ops for writing a 4KB block

recover from log after a crash



Data Journaling

- **Journal write** - write the transaction to the journal
  - transaction-begin block
  - pending data and metadata updates
  - wait for the writes to complete
- **Journal commit**
  - write transaction-end block
  - wait for the writes to complete

make transaction-end write a separate step to ensure integrity of the transaction

- **Checkpoint** - write the pending metadata and data updates to the disk

Recovery

- **Journal write** - write the transaction to the journal
  - transaction-begin block
  - pending data and metadata updates
  - wait for the writes to complete

if a crash happens before the transaction is fully journaled -> the update is **invalid** and will be **skipped**

- **Journal commit**
  - write transaction-end block
  - wait for the writes to complete

if a crash happens **during checkpoint** -> redo the journaled operations during boot

- **Checkpoint** - write the pending metadata and data updates to the disk

Circular log

- Problem: journal can **grow very large** with time
- Solution: **reuse** space by deleting checkpointed transactions
  - add a journal superblock to keep track of transactions
  - update recovery procedure

Recovery

- **Journal write** - write the transaction to the journal
  - transaction-begin block
  - pending metadata and data updates
  - wait for the writes to complete
- **Journal commit**
  - write transaction-end block
  - wait for the writes to complete
- **Checkpoint**
  - write the pending metadata and data updates to the disk
- **Free**
  - mark the transaction free in the journal superblock

if crash happens before transactions is fully journaled -> the update is invalid and will be skipped

if crash happens during checkpoint -> redo the journaled operations during boot

Recovery

Problem: data is written twice

- **Journal write** - **write** the transaction to the journal
  - transaction-begin block
  - pending metadata and **data** updates
  - wait for the writes to complete

first during the journal write

- **Journal commit**
  - write transaction-end block
  - wait for the writes to complete
- **Checkpoint**
  - **write** the pending metadata and **data** updates to the disk

then again during checkpointing

- **Free**
  - mark the transaction free in the journal superblock

data is usually the largest portion of the write
Solution: journal metadata only

Metadata journaling

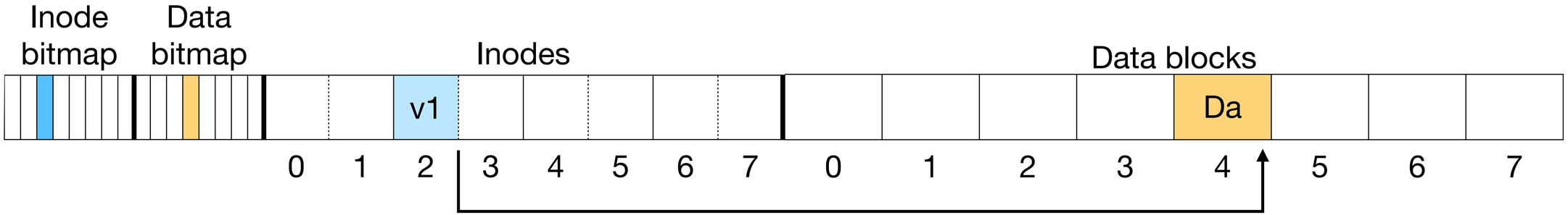
- **Write data to disk**
- **Journal write** - write the transaction to the journal
  - transaction-begin block
  - pending metadata ~~and data~~ updates
  - wait for the writes to complete
- **Journal commit**
  - write transaction-end block
  - wait for the writes to complete
- **Checkpoint**
  - write the pending metadata ~~and data~~ updates to the disk
- **Free**
  - mark the transaction free in the journal superblock

write data to disk first

the rest is the same

Does it matter when we write the data?
should it be written first, last or somewhere in the middle?

create a file
write 4KB



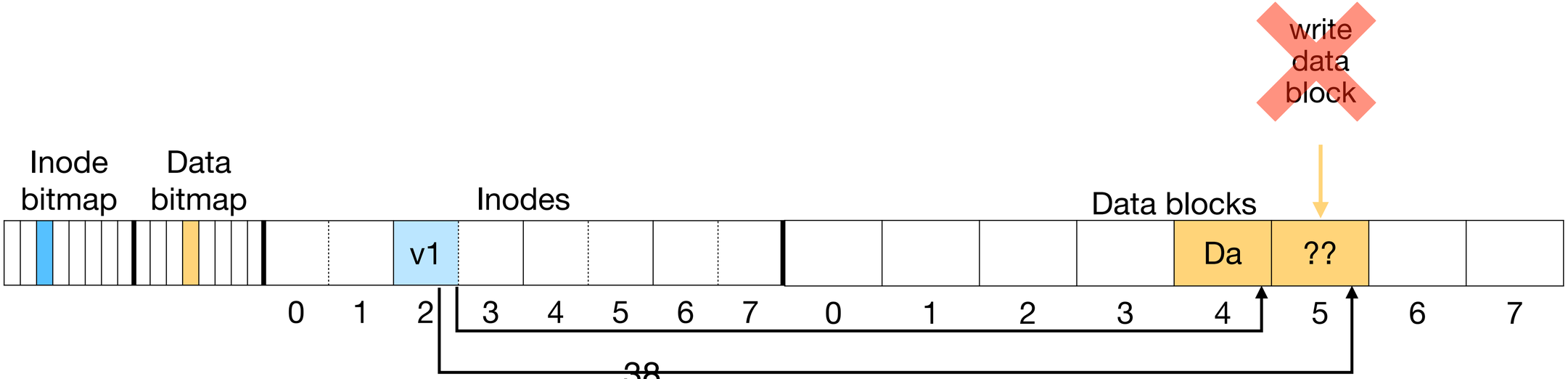
write a log
of all ops for
writing a
4KB block

TxB	update inode	update bitmap	TxE	

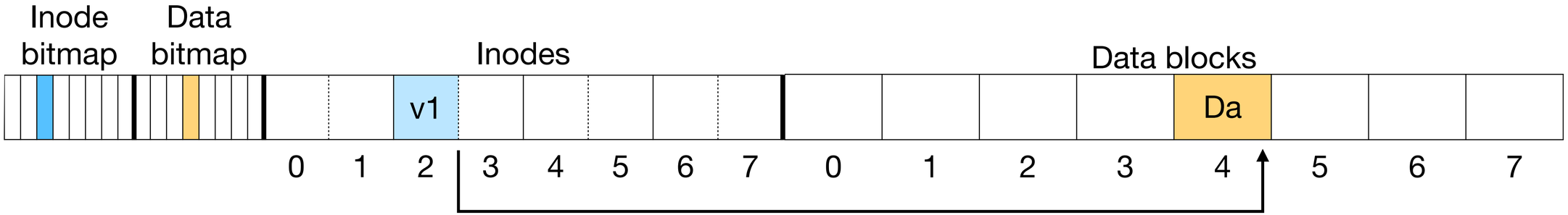
what if we write the metadata first
and data next?

if crash happens during the data write

write
another
4KB block



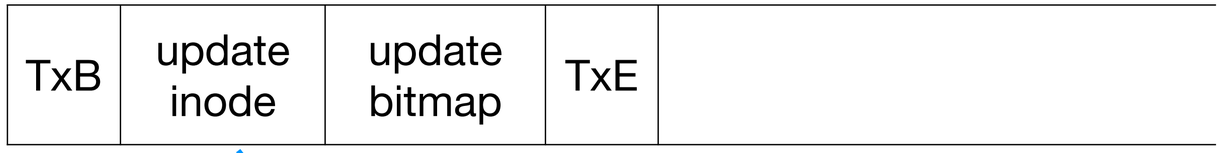
create a file
write 4KB



what if we write the metadata first and data next?

if crash happens during the data write

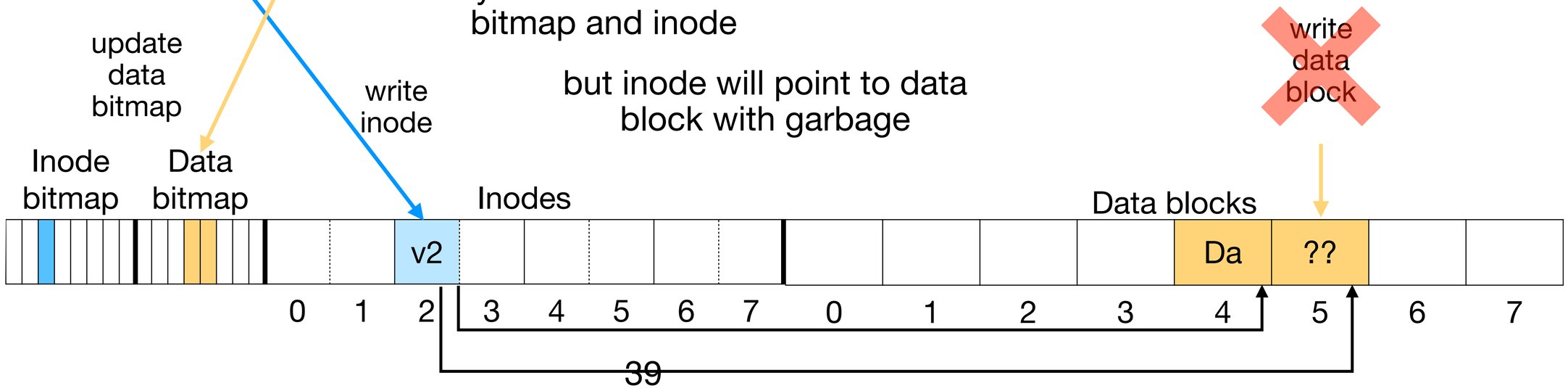
write a log of all ops for writing a 4KB block



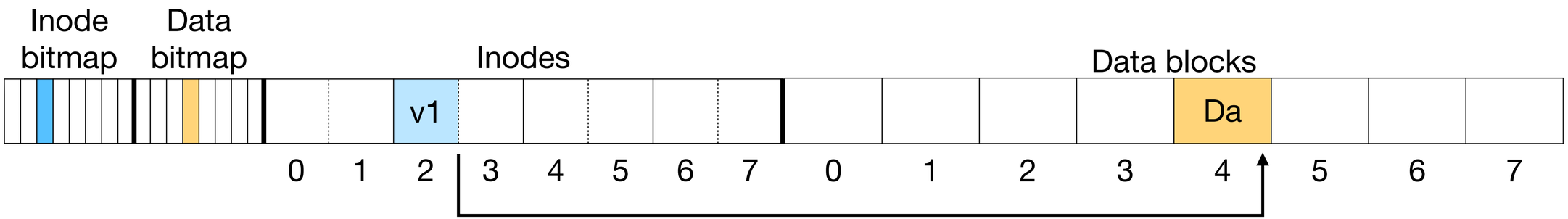
the system will recover the bitmap and inode

but inode will point to data block with garbage

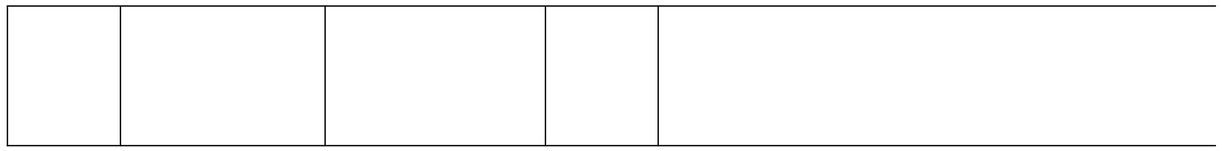
write another 4KB block



create a file
write 4KB



~~write a log of all ops for writing a 4KB block~~

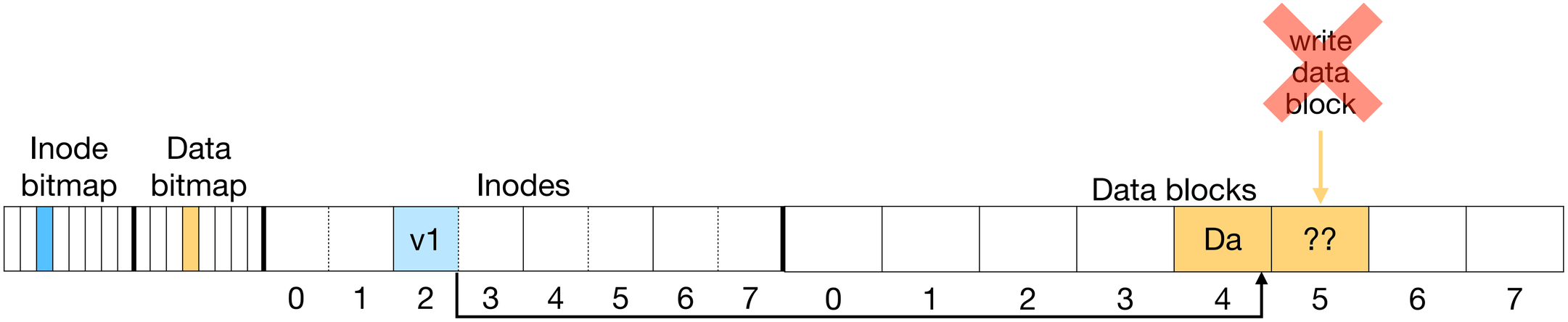


what if we write the data first?

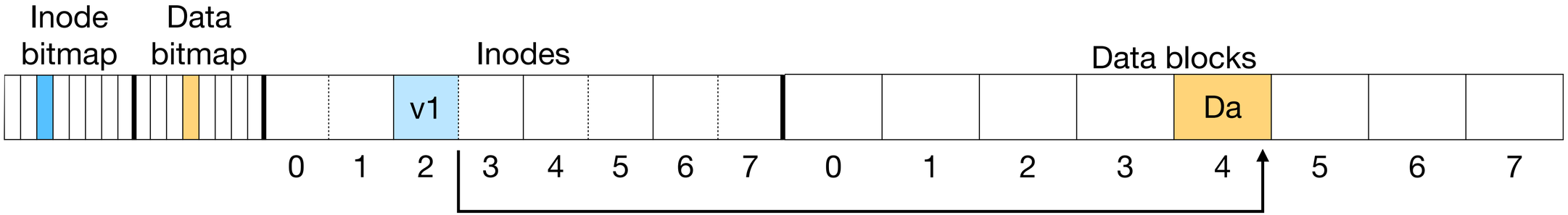
if crash happens during the data write

OK - data loss, no metadata to replay

write
another
4KB block



create a file
write 4KB



~~write a log of all ops for writing a 4KB block~~

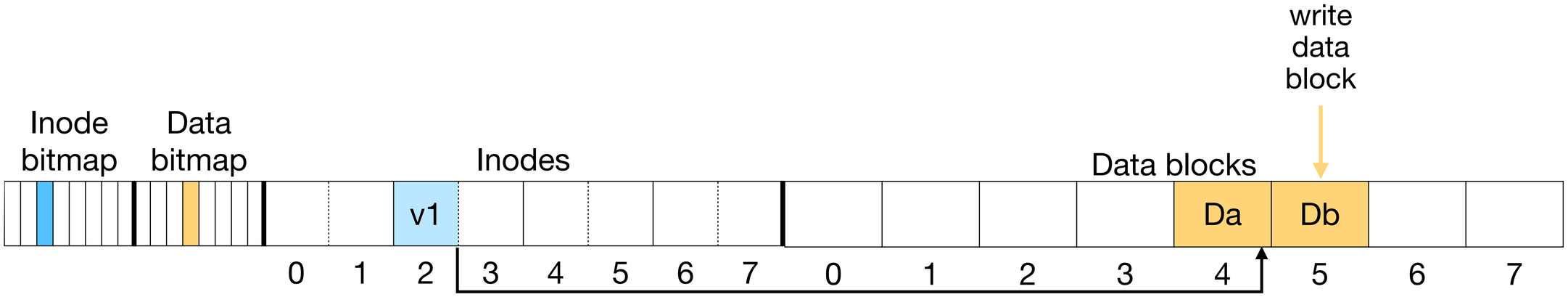
TxB	update inode	??	??	

what if we write the data first?

if crash happens during the metadata write

data loss, consistency OK - no metadata to replay

write
another
4KB block



Metadata journaling

- **Write data to disk**
- **Journal write** - write the transaction to the journal
  - transaction-begin block
  - pending metadata and data updates
  - wait for the writes to complete
- **Journal commit**
  - write transaction-end block
  - wait for the writes to complete
- **Checkpoint**
  - write the pending metadata and data updates to the disk
- **Free**
  - mark the transaction free in the journal superblock

write data to disk first

write the pointed-to object before the object that points to it

prioritize consistency over data loss

metadata must be consistent

Summary so far

- **Journal** - a dedicated space for logging writes
  - prevent file system inconsistencies
  - make recovery more efficient
- **Data journaling**
  - write both data and metadata to journal
  - write both data and metadata to disk

data is larger than metadata

expensive I/O for every write operation

- **Metadata journaling**
  - write data to disk
  - write metadata to journal
  - write metadata to disk

data is written first

avoids pointers to garbage in case of a crash

fewer I/O operations

Summary so far

1. Journal write

data and metadata can be written in any order

Data journaling timeline

Journal				File system	
TxB	metadata	data	TxE	metadata	data
issue	issue	issue			
complete	complete	complete			
complete	complete	complete			
complete	complete	complete			

wait for all previous writes to complete

Summary so far

1. Journal write

data and metadata can be written in any order

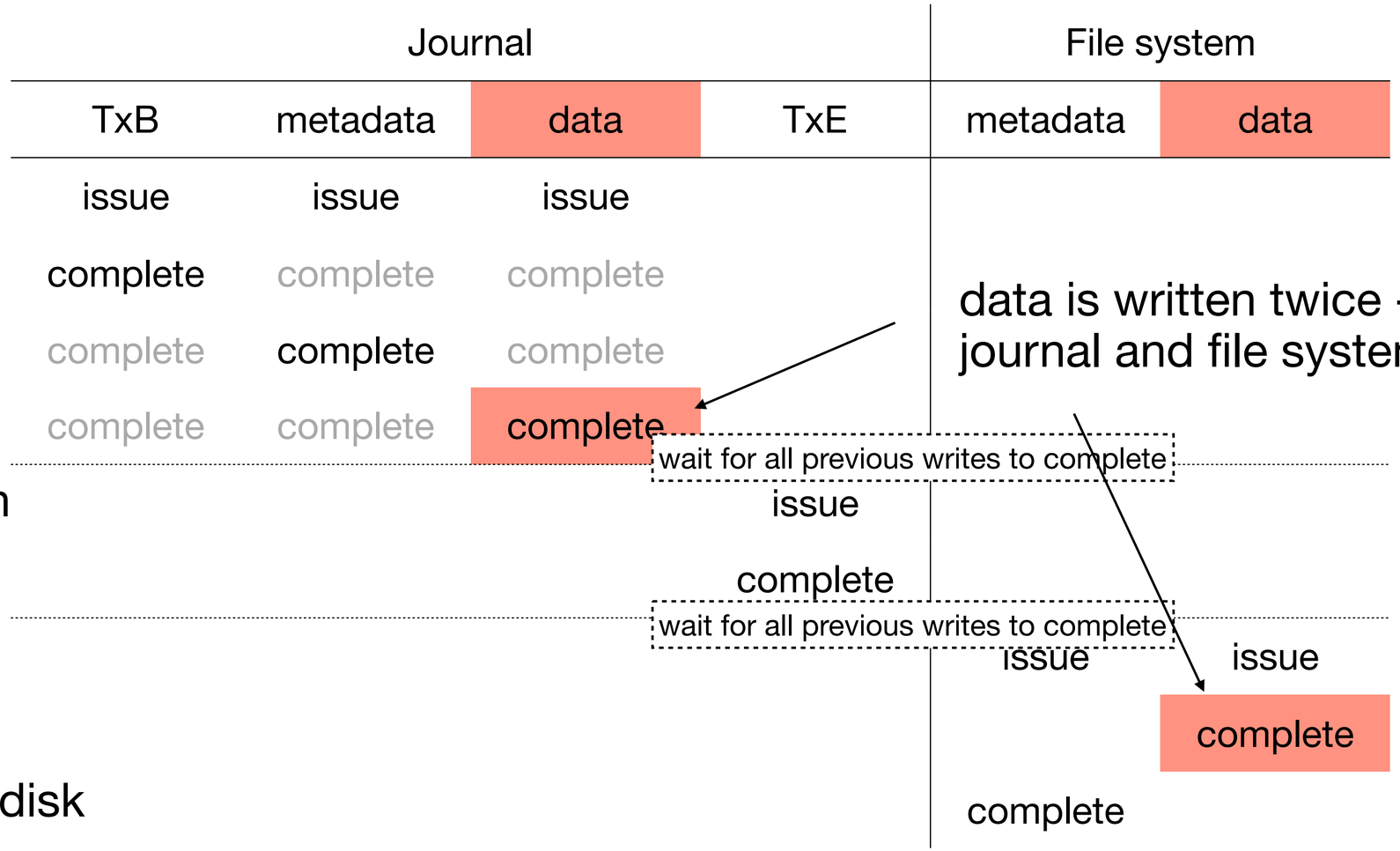
2. Journal commit

the last bit of the transaction can be written only after the journal write is complete

3. Checkpoint

write metadata and data to disk

Data journaling timeline



data is written twice - to journal and file system

Summary so far

1. Journal write

~~data~~ and metadata can be written in any order

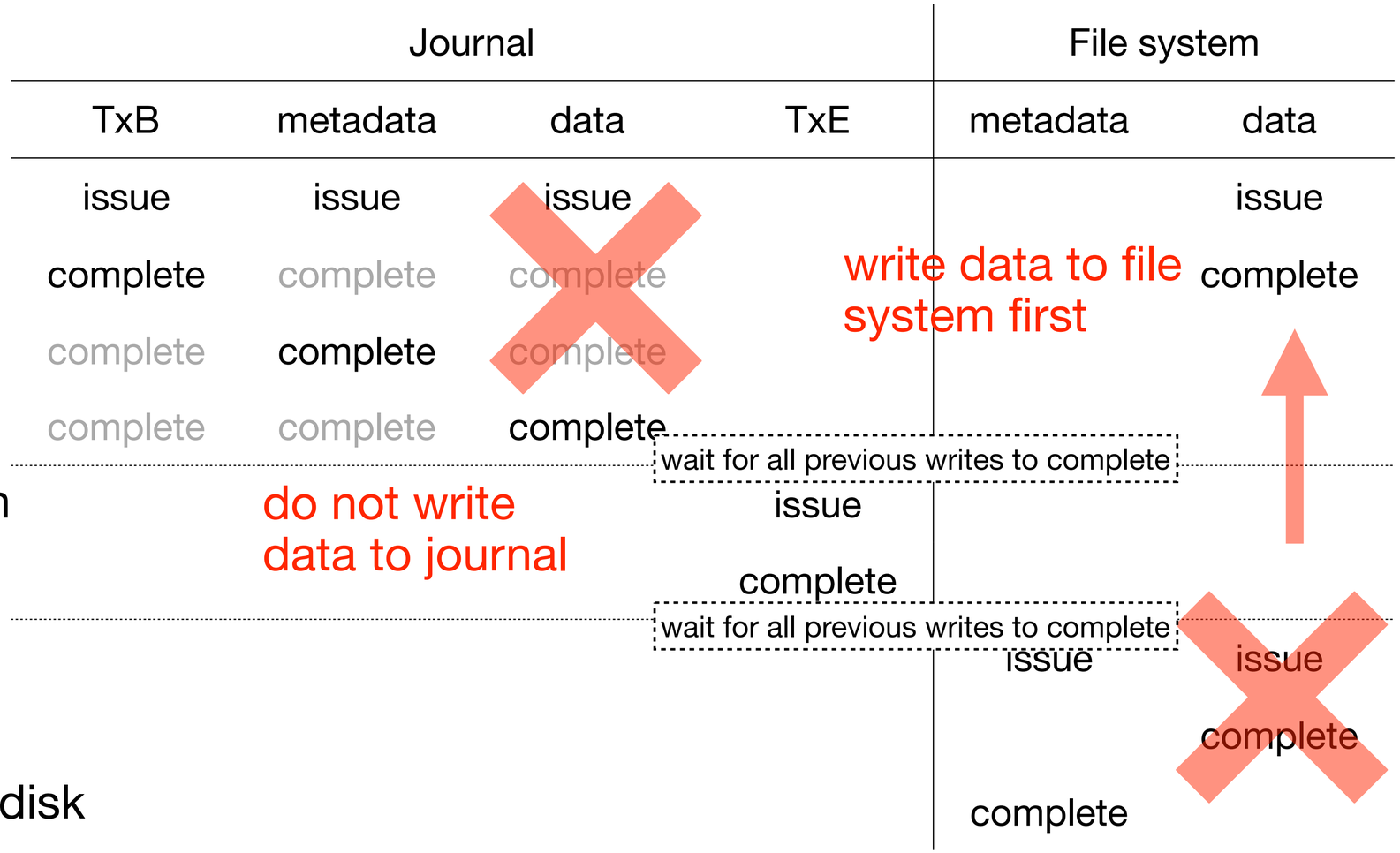
2. Journal commit

the last bit of the transaction can be written only after the journal write is complete

3. Checkpoint

write metadata and ~~data~~ to disk

Metadata journaling timeline



Summary so far

0. Data write to disk

1. Journal write

metadata can be written in any order

Metadata journaling timeline

Journal			File system	
TxB	metadata	TxE	metadata	data
issue	issue			issue
complete	complete			complete
complete	complete			

wait for all previous writes to complete

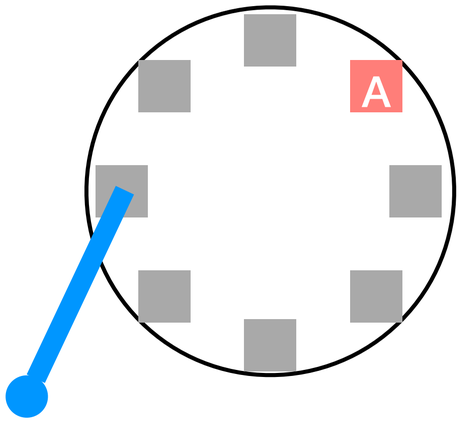
FFS is not fast enough

- **common operations are costly**
  - FFS makes **many reads/writes** for creating/reading files
- gap between random and sequential I/O
  - density of bits on the surface of the HD increased
  - **seek and rotation speed did not** - sequential I/O is still a lot faster
- **larger RAM** can cache more reads -> disk traffic is mostly writes

Effective sequential writes

T=0

write to block A

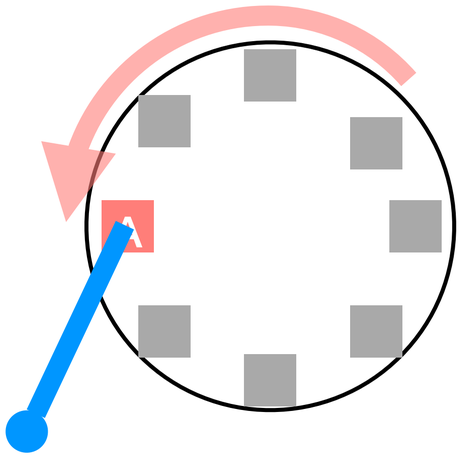


even within the same cylinder group writes can be expensive

Effective sequential writes

T=0

write to block A



even within the same cylinder group writes can be expensive

Effective sequential writes

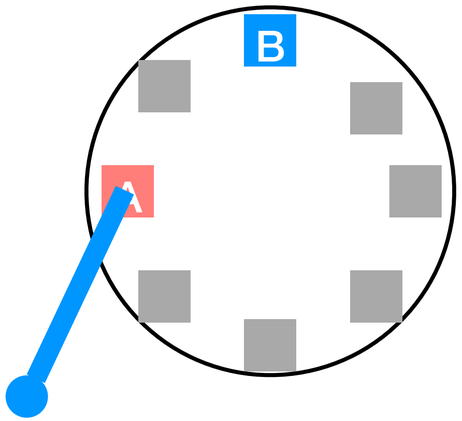
even within the same cylinder group writes can be expensive

T=0

write to block A

T=1

write to block B



Effective sequential writes

even within the same cylinder group writes can be expensive

T=0

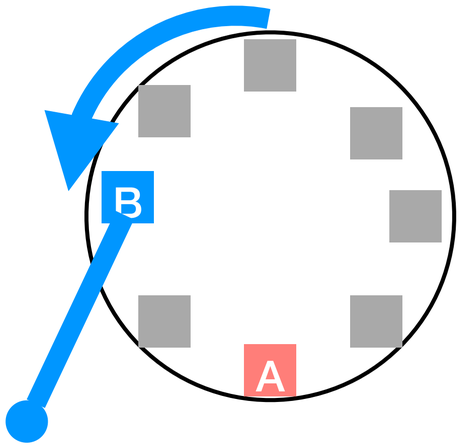
write to block A

T=1

write to block B

T=2

write to block C



Effective sequential writes

even within the same cylinder group writes can be expensive

T=0

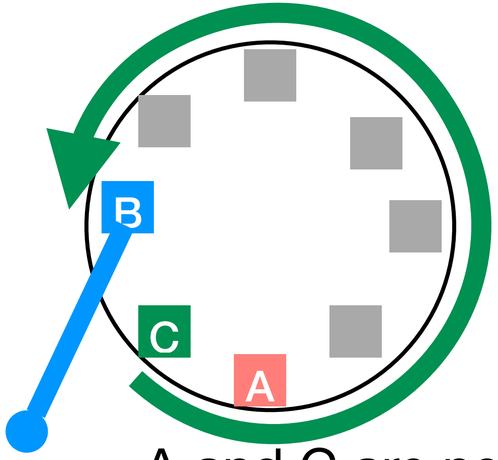
write to block A

T=1

write to block B

T=2

write to block C



A and C are next to each other but it takes almost a full rotation

Effective sequential writes

- Effective writes must be
  - spatially and temporally local
- **Uninterrupted longs writes** are the best for performance

1 seek and 1 rotation

T=0

buffer

write to block A

T=1

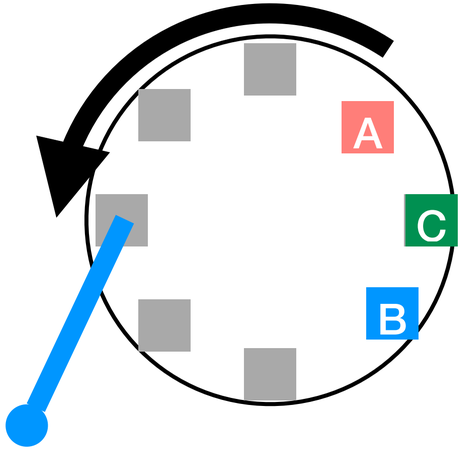
buffer

write to block B

T=2

buffer

write to block C



T = 3

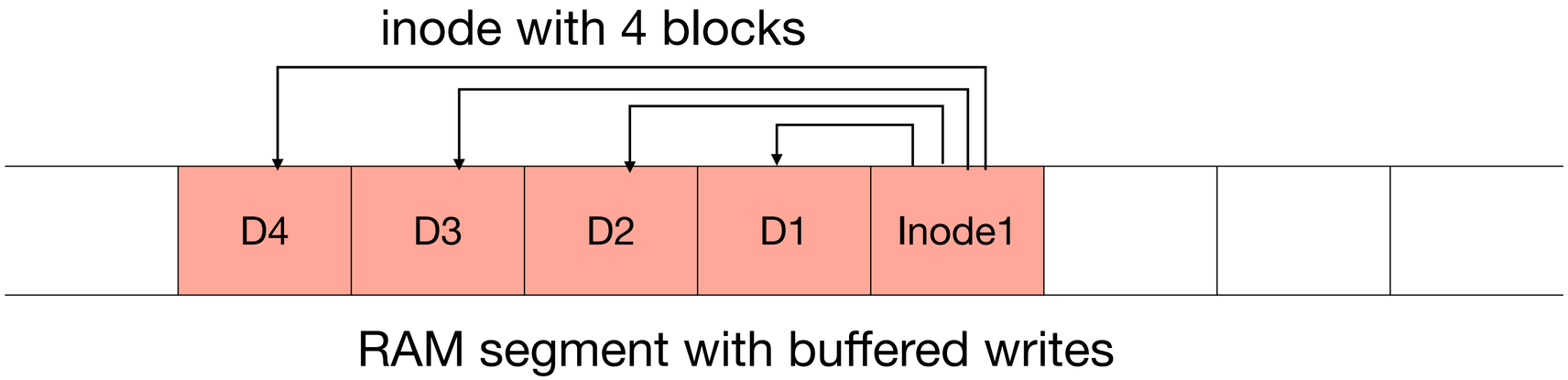
write blocks A, B, C together

LFS

- aka Log-Structured File System
- **Buffer** disk updates (data+metadata) in chunks (**segments**)
- When a segment is full
  - write it in one **long sequential** transfer
  - to an **unused** part of the disk
    - no in-place writes!
- **The entire file system is a log - hence the name**
- Problem #1 - how to organize updates in a segment?
- Problem #2 - how much to buffer?

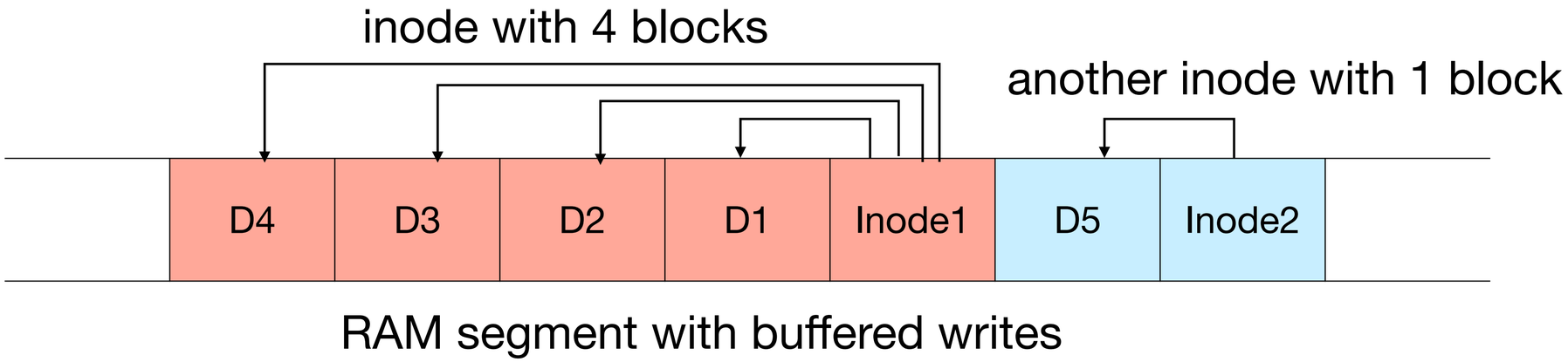
LFS

- Write buffering
  - accumulate memory updates in a RAM segment
  - once the segment is full
  - write segment to the disk



LFS

- Write buffering
  - accumulate memory updates in a RAM segment
  - once the segment is full
  - write segment to the disk



LFS

- **How much to buffer?**
- Depends on the hard drive
  - Each write has a positioning latency $T_{position}$ sec
  - Transfer speed depends on the drive
  - Need to transfer enough data to justify the positioning cost

LFS

- **How much to buffer?**
- Depends on the hard drive
  - Each write has a positioning latency $T_{position}$ sec
  - Transfer speed depends on the drive
  - Need to transfer enough data to justify the positioning cost

$$T_{write} = T_{position} + \frac{D}{R_{peak}}$$

data (MB)

peak transfer rate (MB/s)

time to write

positioning delay

time to transfer D amount of data at peak rate

e.g. Tposition = 0.01 s R peak = 100 MB/s D = 100 MB

LFS

- **How much to buffer?**
- Depends on the hard drive
  - Each write has a positioning latency $T_{position}$ sec
  - Transfer speed depends on the drive
  - Need to transfer enough data to justify the positioning cost

$$T_{write} = T_{position} + \frac{D}{R_{peak}}$$

data (MB)

peak transfer rate (MB/s)

time to write

positioning delay

time to transfer D amount of data at peak rate

$$R_{effective} = \frac{D}{T_{write}}$$

amount of data divided by time it takes to write it (positioning+transfer)

effective rate of transfer

e.g. Tposition = 0.01 s

R peak = 100 MB/s

D = 100 MB

R = 100 / 1.01 = 99 MB/s

LFS

$$T_{write} = T_{position} + \frac{D}{R_{peak}} \qquad R_{effective} = \frac{D}{T_{write}}$$

$$R_{effective} = \frac{D}{T_{write}} = F \times R_{peak}$$

effective rate should be a fraction F of the peak rate

F is between 0 and 1

we want F to be closer to 1

how much data do we need to transfer?

$$R_{effective} = \frac{D}{T_{write}} = 0.99 \times 100MB/s$$

e.g. peak rate is 100 MB/s

we want 99% of that

LFS

$$T_{write} = T_{position} + \frac{D}{R_{peak}} \qquad R_{effective} = \frac{D}{T_{write}}$$

$$R_{effective} = \frac{D}{T_{write}} = F \times R_{peak}$$

effective rate should be a fraction F of the peak rate

F is between 0 and 1

we want F to be closer to 1

how much data do we need to transfer?

$$R_{effective} = \frac{D}{T_{write}} = \frac{D}{T_{position} + \frac{D}{R_{peak}}} = 0.99 \times 100MB/s$$

e.g. peak rate is 100 MB/s

we want 99% of that

LFS

$$T_{write} = T_{position} + \frac{D}{R_{peak}} \qquad R_{effective} = \frac{D}{T_{write}}$$

$$R_{effective} = \frac{D}{T_{write}} = \frac{D}{T_{position} + \frac{D}{R_{peak}}} = F \times R_{peak}$$

effective rate should be a fraction F of the peak rate

F is between 0 and 1

we want F to be closer to 1

solve for D

$$D = F \times R_{peak} \times (T_{position} + \frac{D}{R_{peak}})$$

open the brackets

$$D = (F \times R_{peak} \times T_{position}) + (F \times \cancel{R_{peak}} + \frac{D}{\cancel{R_{peak}}})$$

move Ds to the left

$$D - FD = R_{peak} \times T_{position} \times F$$

solve for D

$$D = \frac{F}{1 - F} \times R_{peak} \times T_{position}$$

LFS

fraction of the
peak bandwidth

positioning latency

$$D = \frac{F}{1-F} \times R_{peak} \times T_{position}$$

peak transfer rate

hard drive specs

Tposition = 10 ms
Rpeak = 100 MB/s
F = 0.9

substitute into the equation

$$D = \frac{0.9}{1-0.9} \times 100MB/s \times 0.01sec = 9MB$$

size of the cache for 90%
peak transfer rate

what if we want F=0.99?

$$D = \frac{0.99}{1-0.99} \times 100MB/s \times 0.01sec = 99MB$$

size of the cache for 99%
peak transfer rate

LFS

- **How much to buffer?**
- Calculate buffer size D for any given hard drive based on
  - peak transfer rate
  - positioning latency
  - desired fraction of the peak transfer rate

fraction of the
peak bandwidth

positioning latency

$$D = \frac{F}{1 - F} \times R_{peak} \times T_{position}$$

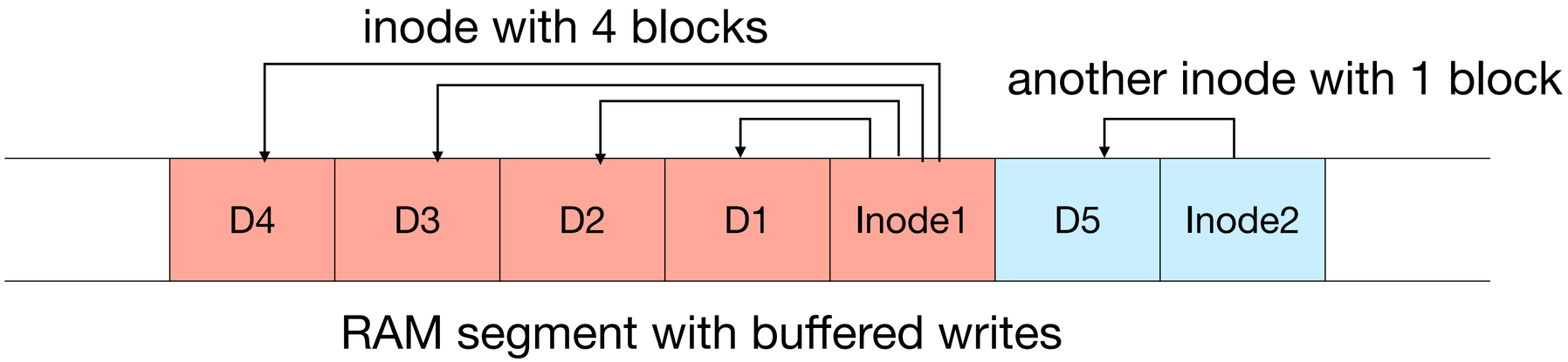
peak transfer rate

LFS

- Write buffering
  - accumulate memory updates in a RAM segment
  - once the segment is full
  - write segment to the disk

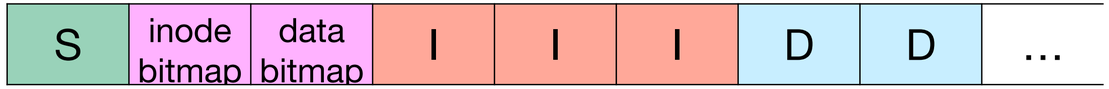
optimal segment size for the desired transfer rate

how do we find inodes after writing them to disk?



How to find inodes

vsfs



superblock stores location of the inode table

inode is an offset into inode table

data blocks are linked from inodes

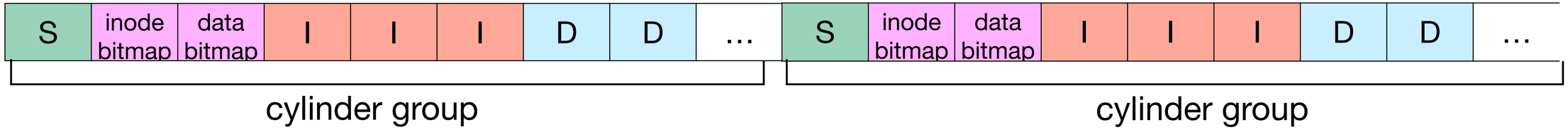
FFS duplicates vsfs structure in every cylinder group

Superblock at the beginning of each group stores group data

if inode is not in this group, go to the next group

the rest is the same

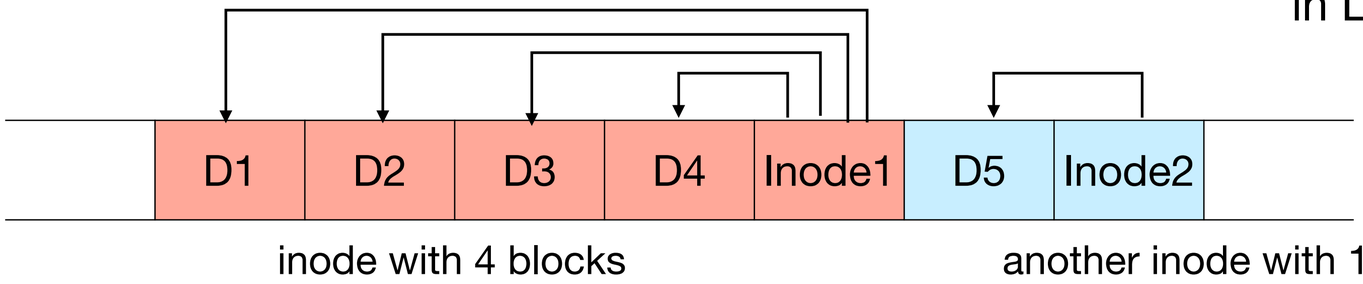
FFS



in LFS inodes are scattered across the drive

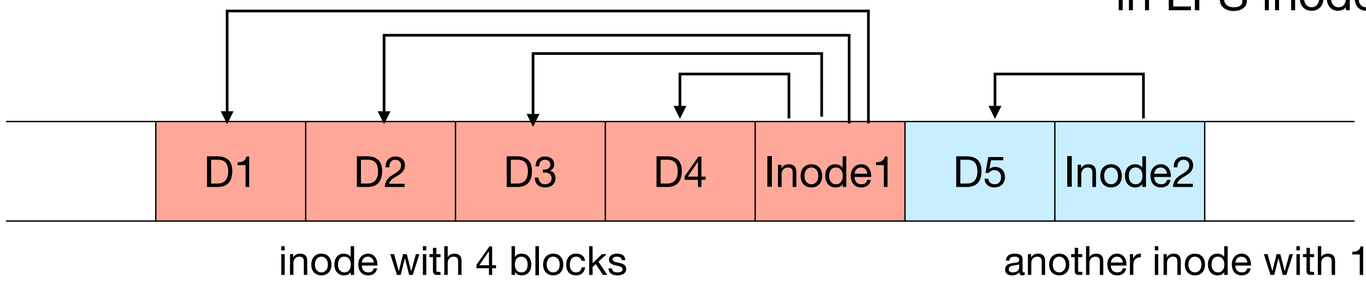
Problem: how can we find an inode for any file without scanning the whole drive?

LFS



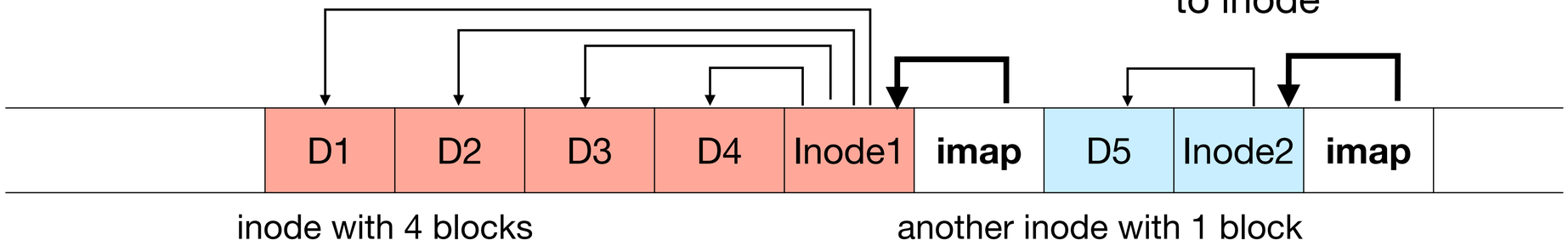
LFS inode map

in LFS inodes are scattered across the drive



Problem: how can we find an inode for any file without scanning the whole drive?

Solution: write an inode map with the link to inode

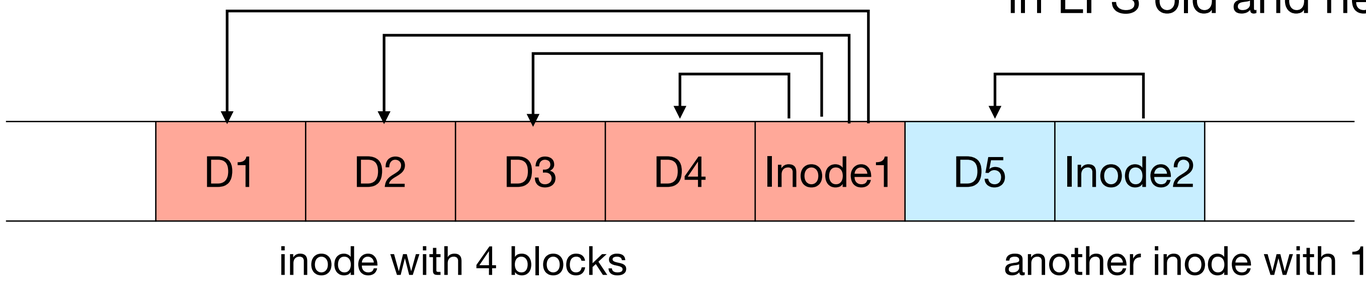


Problem: how can we find the imaps?

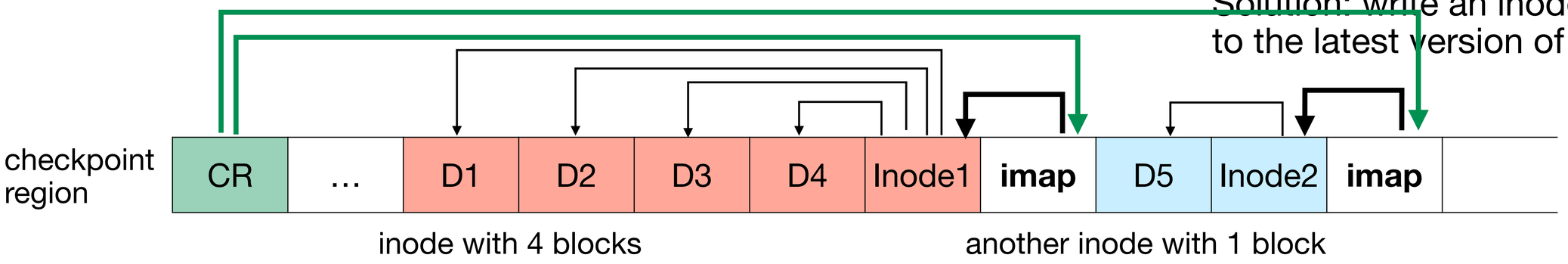
LFS inode map

in LFS old and new inodes are scattered across the drive

Problem: how can we find an inode for any file without scanning the whole drive?



Solution: write an inode map with the link to the latest version of inode



Problem: how can we find the imaps?

Solution: add a checkpoint region (CR) at a fixed location on the disk with links to the pieces of inode map

LFS inode map

- **Checkpoint region** at a fixed location on the drive
  - contains **pointers** to the pieces of inode map
    - a system-wide table of all files and directories
  - inode map points to the most recent version of inode
  - inode points to the data
    - via direct/indirect pointers
- **Checkpoint region is updated rarely** (every 5-30 s) to reduce I/O overhead

LFS finding an inode

- Read the checkpoint region
  - Access all imaps and cache them in RAM
- Look up inode in imap
- Follow the link to the most recent version
- Follow the block links (direct or indirect) from inode to its data

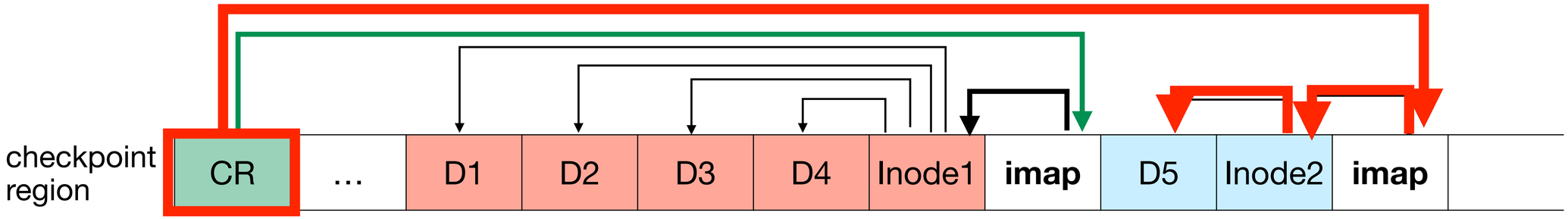
accessing inode2

cache CR in RAM

find imap that has inode2

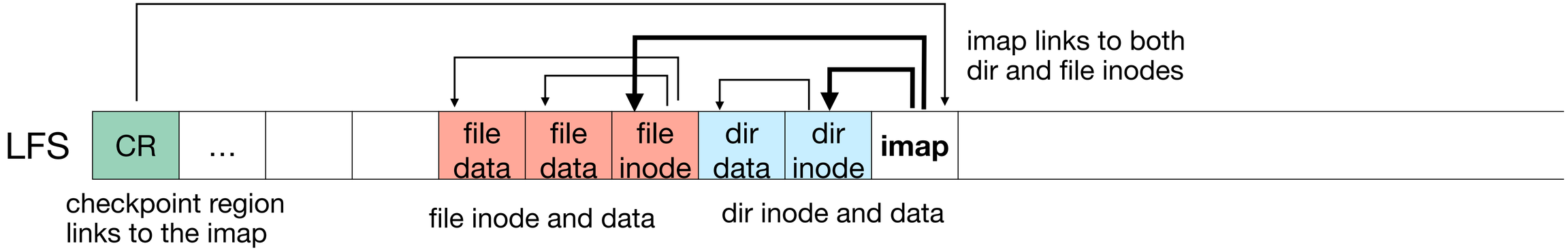
follow the link from imap to inode2

follow the link from inode2 to data

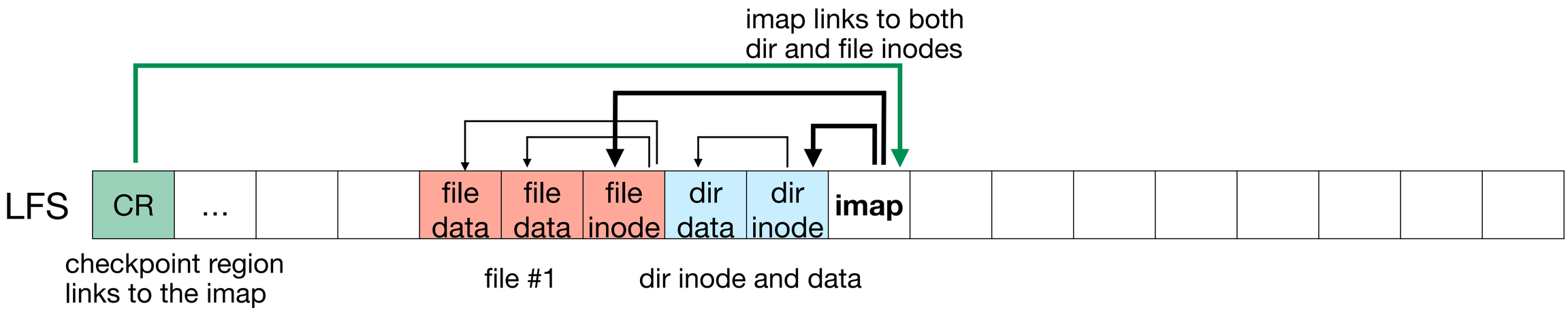


LFS directories

- **Directories are treated same way as files**
  - Imap points to the directory inode
  - Directory inode points to directory data
  - Directory data lists names and inodes for files in the directory

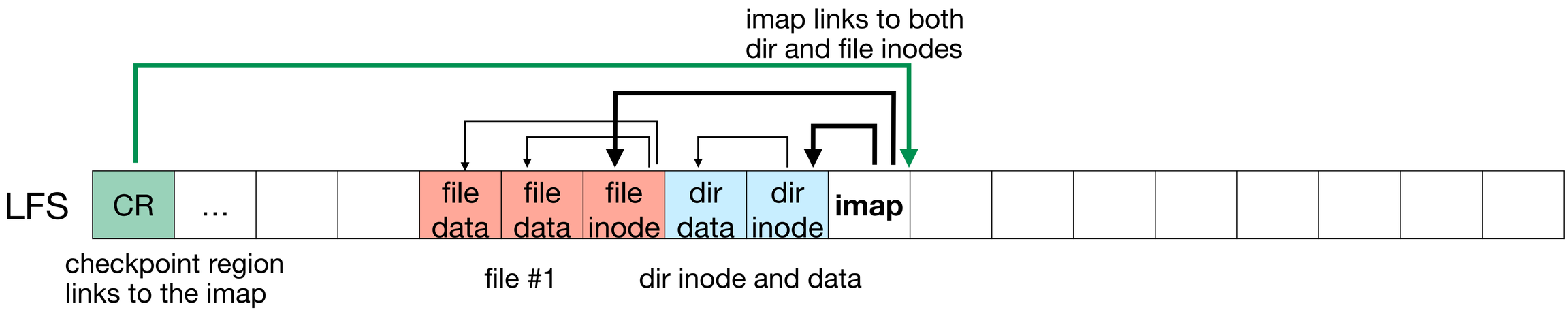


LFS directories



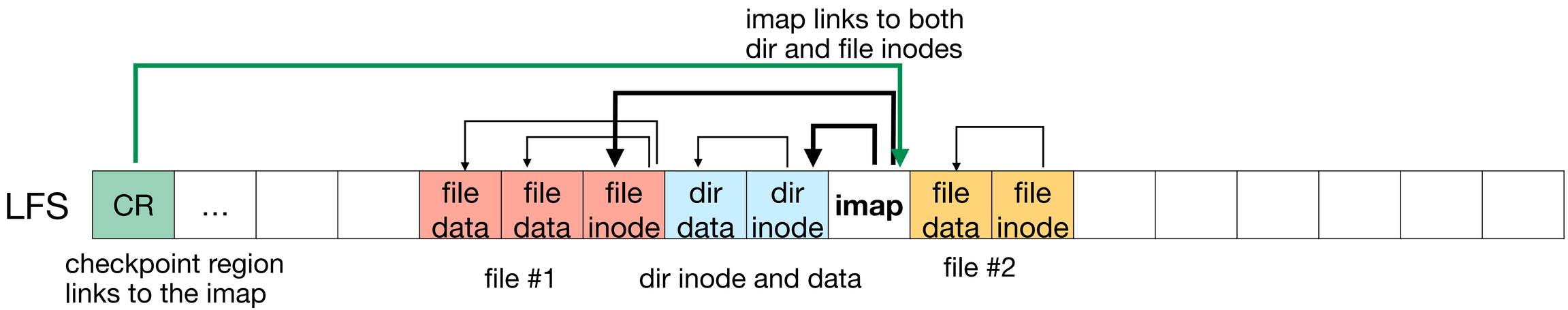
what if we add another file to the directory?

LFS directories



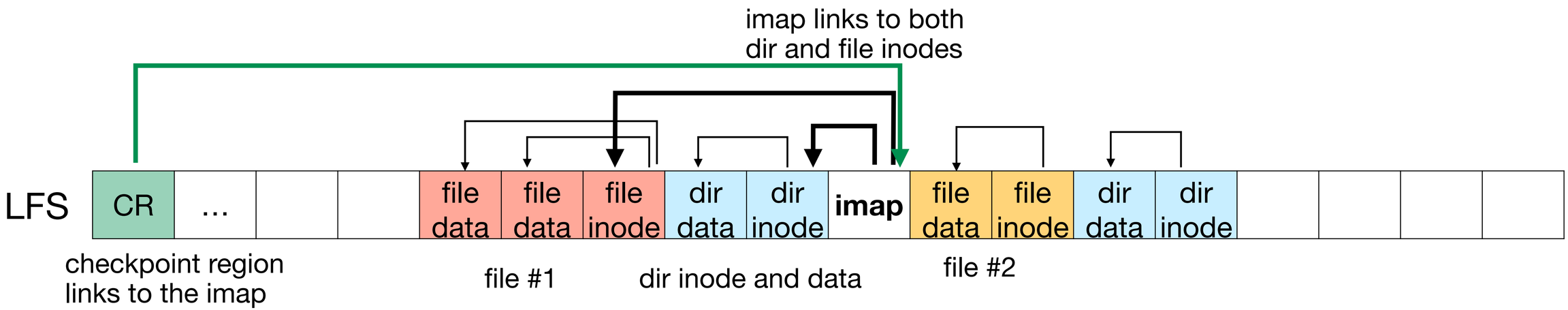
what if we add another file to the directory?

LFS directories



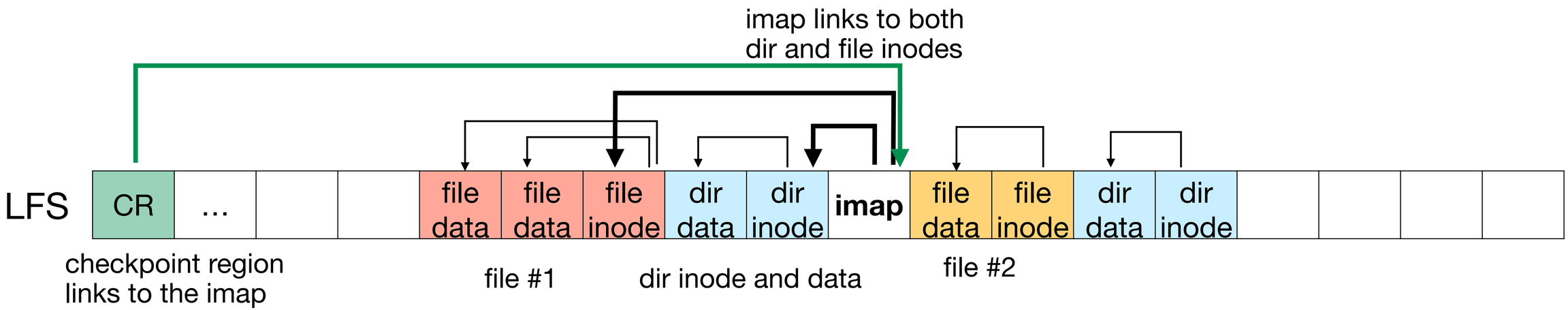
what if we add another file to the directory?
write file #2 and update dir inode

LFS directories



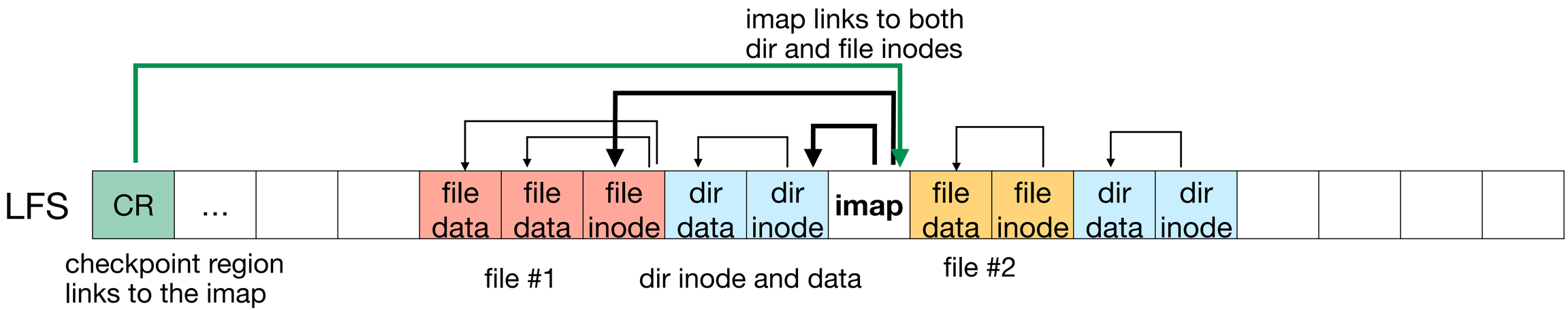
what if we add another file to the directory?
add inode for file#2 to dir data
write file #2 and updated dir inode

LFS directories



what if we add another file to the directory?
add inode for file#2 to dir data
write file #2 and updated dir inode

LFS directories



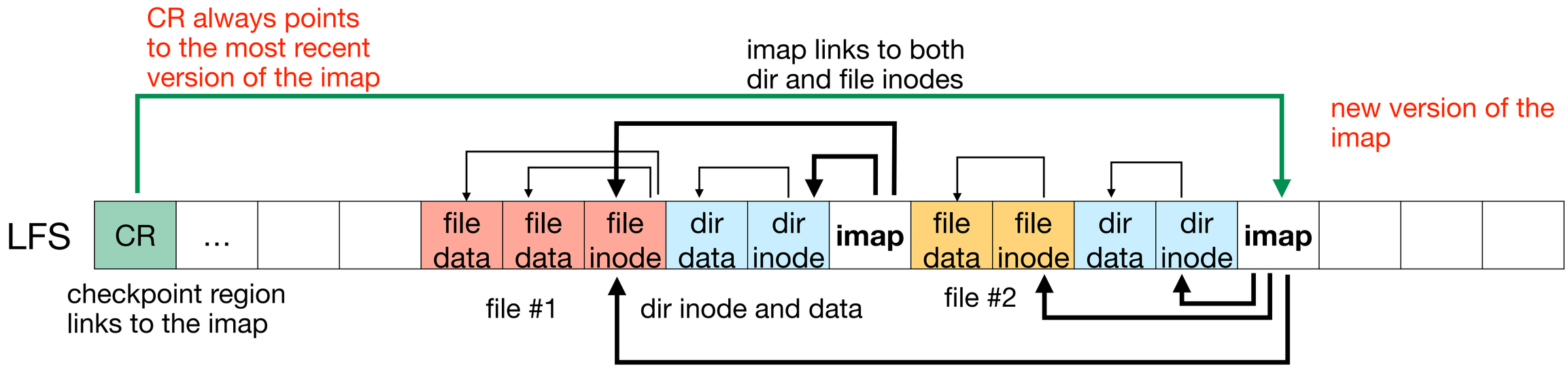
what if we add another file to the directory?

add inode for file#2 to dir data

write file #2 and updated dir inode

inode for file #1 is the same

LFS directories



what if we add another file to the directory?

add inode for file#2 to dir data

write file #2 and updated dir inode

inode for file #1 is the same

imap points to the new dir inode, file #1 inode and file #2 inode

CR points to the updated version of the imap

Crash recovery

- Read the most recent valid checkpoint
  - Checkpoint contains addresses of all inode maps
  - Initialize inode table in RAM from inode maps
- Roll-forward through segments from the last segment in the checkpoint
  - Update the in-memory inode map with recovered inodes

Summary

- **Crash consistency**
  - vsfs and FFS have poor crash consistency
  - crashes can cause metadata issues and data loss
- **fsck - a passive solution**
  - scan the hard-drive and rebuild the file system
  - cons: takes a very long time to do
- **journaling - an active solution**
  - write data/metadata to a log first
  - pros: don’t need to scan the whole hard drive

Summary

- **Log-structured file system (LFS)**
  - improves both write efficiency and crash recovery
  - the whole file system is a log -> no in-place writes!
  - updates the drive every 30 s
    - at most 30 s data loss

Reading

- Chapter 42 FSCK and Journaling
  - https://pages.cs.wisc.edu/~remzi/OSTEP/file-journaling.pdf
- Chapter 43 Log-structured File System (LFS)
  - https://pages.cs.wisc.edu/~remzi/OSTEP/file-lfs.pdf

Final exam

- 2 hours duration
- In-person, closed book, no aids (e.g. any electronics, calculators, etc.)
- The same format as midterm but longer
  - 60% - 30 multiple-choice questions (2 points each)
  - 40% - 8 short-answer questions (5 points each)
    - crossword, open-ended questions, filling the blanks, etc.
- Topics covered
  - 30% on pre-midterm material (memory and CPU virtualization)
  - 70% on post-midterm material (concurrency and persistence)

How to study

- General advice
  - **Avoid all-nighters**
    - Sleep **consolidates** memories - you will remember stuff **better** if you slept on it
  - Attend lectures/labs/review sessions
    - **Extra exposure** to the material, **extra practice** never hurts
  - Be **strategic** about your grades

How to study

- Exam prep
  - Write down **what to study** and **how long it will take** to review
  - Reading and understanding **IS NOT** the same as replicating
    - Say, you read a cache tracing sequence and **think you know** how it works
    - Test yourself: try doing it by hand on paper **without looking at the slides**
    - If you cannot, you will **likely not be able to do it** during the test
  - Study **on your own first** and then with others
    - Study with a mix of **stronger** and **weaker** students than you
  - **Start early** - you will need **at least a week** to prepare
    - likely more if you did not study during the term

How to study

- Use your time with purpose and maximize opportunities to grow
  - Focus on **real-world experience**
    - **For industry** - do side projects, contribute to open-source
    - **For grad school** - look for research opportunities
  - **Network** (aka talk to people)
    - Referrals and reference letters matter

CIS*4900/4910

- **If you want to give this a try**
  - Check **eligibility** (7.00 credits in CIS + instructor consent)
  - **Contact** potential supervisor(s)
    - I'll be happy to talk to anyone interested, you know where to find me

THE LAST QUIZ 20

- This quiz **is graded**
- Your Kahoot! nickname **must be your student id**
  - e.g. 9999999
- Use any device, e.g. phone, laptop, tablet
- If you get disconnected and cannot use the same id, append underscore (_) before the id, e.g. _9999999